

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONICS ENGINEERING



CAPSTONE PROJECT 2

**Design and Implementation of a
RISC-V Vector Processing Unit**

Supervisor: Dr. Nguyen Tuan Hung

Student: Pham Anh Khoi

Student ID: 2251026

HO CHI MINH CITY, JUNE 2025

Declaration of Originality

I hereby declare that this thesis, “*Design and Implementation of a RISC-V Vector Processing Unit Targeting ASIC Tapeout*,” is my own original work carried out under the supervision of TS. Nguyễn Tuấn Hùng at Ho Chi Minh City University of Technology. All sources of information used in this work are duly acknowledged and referenced. No part of this thesis has been submitted for any other degree or qualification at this or any other institution.

Ho Chi Minh City, June 2025

Phạm Anh Khôi

Acknowledgements

I would like to express my sincere gratitude to my supervisor, TS. Nguyễn Tuấn Hùng, for his invaluable guidance, encouragement, and technical insight throughout this project. His expertise in digital design and computer architecture continually challenged me to think rigorously and produce work of the highest quality.

I am also grateful to the faculty and staff of the Department of Computer Science and Engineering at Ho Chi Minh City University of Technology for providing an excellent educational environment. Special thanks go to my classmates and lab partners for the many stimulating discussions that shaped both the design and this report.

Finally, I thank my family for their unwavering support and patience during the long nights of debugging and writing. This work would not have been possible without them.

Abstract

This thesis presents the design and implementation of a vector processing unit (VPU) integrated with a RISC-V scalar core, targeting ASIC tapeout. The system implements the `rv32im_zicsr_zve32x_zvl128b` instruction set architecture, providing a 128-bit vector register file with support for element widths of 8, 16, and 32 bits and register grouping factors (LMUL) of 1, 2, 4, and 8.

The VPU is implemented as a single-lane, multi-cycle execution engine controlled by a finite state machine. It supports the full Zve32x subset including arithmetic, logical, shift, compare, reduction, and vector load/store operations, totalling more than 50 distinct instruction encodings. A five-stage pipelined RISC-V scalar core (RV32IM with load-use stall and data forwarding) dispatches vector instructions to the VPU through a dedicated instruction queue. The scalar-vector interface follows an occupancy-based stall protocol to maintain correct program order.

The design is verified using ModelSim with a comprehensive testbench suite of 172 directed test cases, all passing. Integration-level benchmarks include BT.601 RGB-to-grayscale conversion on a 128×128 image (16 384 pixels verified correct) and matrix multiplication. A UART interface enables real-data end-to-end testing. FPGA prototyping on the DE10-Standard (Intel Cyclone V) achieves an $F_{\max}$ of 47.18 MHz at 10 438 adaptive logic modules, representing a 25% area reduction and $3\times$ frequency improvement after redesigning the reduction unit from a 64-deep combinational chain to a serial per-element accumulator.

Keywords: RISC-V, vector processor, RVV, Zve32x, ASIC design, SIMD, image processing, FPGA prototyping.

Contents

List of Abbreviations	viii
1 Introduction	1
1.1 Motivation and Background	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Contributions	4
1.5 Thesis Organisation	5
2 Background and Related Work	7
2.1 RISC-V Instruction Set Architecture	7
2.1.1 History and Design Philosophy	7
2.1.2 Base ISA: RV32I	7
2.1.3 M Extension: Integer Multiplication and Division	8
2.1.4 Privileged Specification and CSRs	8
2.2 Vector Processing Fundamentals	8
2.2.1 Flynn's Taxonomy and Parallelism Classes	8
2.2.2 Key Vector Concepts	9
2.2.3 Performance Advantages of Vector Processing	10
2.3 RISC-V Vector Extension (RVV)	10
2.3.1 Overview and Design Goals	10
2.3.2 Vector CSRs	10
2.3.3 Vector Register File	11
2.3.4 Instruction Categories	11
2.3.5 Zve32x Subset	11
2.4 ASIC Design Methodology	12
2.4.1 RTL Design Phase	12
2.4.2 Logic Synthesis	12
2.4.3 Static Timing Analysis	12
2.4.4 Place and Route	13
2.4.5 Physical Verification: DRC and LVS	13
2.5 Related Work	13
2.5.1 Ara: Scalable RVV Vector Processor in 22nm FD-SOI	13
2.5.2 CVA6 / Ariane	14
2.5.3 Vitruvius+	14
2.5.4 Hwacha	14

2.5.5	Comparative Analysis	14
2.6	Summary	15
3	System Specification	16
3.1	System Requirements	16
3.1.1	Functional Requirements	16
3.1.2	Non-Functional Requirements	17
3.2	ISA Specification	17
3.2.1	Instruction Encoding Conventions	17
3.2.2	Supported Instructions by Category	18
3.3	Vector Configuration	20
3.3.1	VLEN and Register Geometry	20
3.3.2	SEW Encoding	20
3.3.3	LMUL Encoding	20
3.3.4	VLMAX Formula	21
3.3.5	vtype Register Layout	21
3.4	Memory Architecture	22
3.4.1	Harvard Architecture Overview	22
3.4.2	Instruction Memory (IMEM)	23
3.4.3	Data Memory (DMEM)	24
3.4.4	Address Decode	24
3.5	UART Interface Specification	25
3.5.1	Physical Framing	25
3.5.2	MMIO Register Map	25
3.5.3	Firmware Programming Model	26
3.5.4	Timing Constraints	27
3.6	Performance Requirements	27
3.6.1	Target Clock Frequency	27
3.6.2	Area Budget	27
3.6.3	Power Estimation	28
4	Hardware Design	29
4.1	System Architecture Overview	29
4.1.1	Top-Level Integration	29
4.1.2	Clock and Reset Distribution	30
4.1.3	Signal Interface Summary	31
4.2	Scalar RISC-V Core	31
4.2.1	Pipeline Structure	32
4.2.2	Hazard Detection	33
4.2.3	Data Forwarding	33

4.2.4	VPU Dispatch Protocol	34
4.3	VPU Architecture	34
4.3.1	Instruction Decoder	35
4.3.2	Instruction FIFO	37
4.3.3	Execution FSM	38
4.3.4	Vector Register File	40
4.3.5	VRF Address Generator	40
4.3.6	Execution Lane	41
4.3.7	Arithmetic and Logic Units	43
4.3.8	Reduction Unit	44
4.3.9	Vector Load/Store Unit	46
4.4	UART Controller	49
4.4.1	8N1 Framing	49
4.4.2	MMIO Register Map	50
4.4.3	Integration and TL-UL Interface	50
4.4.4	FIFO Depth and Overflow Behaviour	50
4.5	Memory Subsystem	51
4.5.1	Instruction Memory (<code>imem_sync</code>)	51
4.5.2	Data Memory (<code>dmem_sync</code>)	51
4.5.3	Memory Address Map	52
4.5.4	Harvard Architecture Tradeoffs	52
4.6	Chapter Summary	53
5	Software Design	55
5.1	Development Toolchain and Build System	55
5.1.1	Compiler and Assembler	55
5.1.2	Build Flow	56
5.1.3	The <code>elf2hex_lines.py</code> Converter	57
5.1.4	Makefile Targets	57
5.2	Memory Model and Linker Script	58
5.2.1	Harvard Architecture Implications	58
5.2.2	Linker Script	58
5.2.3	Stack Size and Calling Convention	59
5.3	Firmware Architecture	59
5.3.1	Startup and Initialisation	59
5.3.2	UART Register Map	59
5.3.3	The <code>recv_block</code> Subroutine	60
5.3.4	The <code>uart_tx_byte</code> Subroutine	60
5.3.5	The <code>send_block</code> Subroutine	61

5.4	Application: BT.601 Grayscale Conversion	62
5.4.1	Background: ITU-R BT.601 Luma Formula	62
5.4.2	Mapping to <code>vmulhu.vx</code>	62
5.4.3	Full Instruction Sequence	62
5.4.4	Throughput Analysis	63
5.4.5	Accuracy Analysis	64
5.5	UART Communication Protocol	64
5.5.1	Physical Layer Parameters	64
5.5.2	Image Streaming Protocol	64
5.5.3	Echo and Acknowledgement	65
5.5.4	Timing Budget	65
5.6	Summary	65
6	Verification and Evaluation	67
6.1	Verification Methodology	67
6.1.1	V-Model Approach	67
6.1.2	Simulation Environment	67
6.1.3	Regression Gate Policy	68
6.2	Unit-Level Verification	68
6.2.1	Adder Unit (<code>vproc_adder</code>)	68
6.2.2	Multiplier Unit (<code>vproc_mul</code>)	68
6.2.3	Reduction Unit (<code>vproc_reduction</code>)	69
6.2.4	Vector LSU (<code>vproc_vec_lsu</code>)	69
6.3	Instruction Regression (<code>tb_vproc_all_instr</code>)	70
6.3.1	Test Structure	70
6.3.2	Coverage Table	70
6.3.3	Representative Test Vectors	71
6.4	System Integration Verification	71
6.4.1	End-to-End Image Benchmark (<code>tb_lena_gray</code>)	71
6.5	Performance Analysis	73
6.5.1	VPU Throughput Model	73
6.5.2	Benchmark Cycle Counts	74
6.5.3	Speedup Over Scalar Baseline	74
6.5.4	Bottleneck Identification	74
6.6	FPGA Synthesis and Timing Analysis	75
6.6.1	Target Device and Constraints	75
6.6.2	Final Synthesis Results	75
6.6.3	Optimisation History	76
6.6.4	Area Breakdown Estimate	77

6.7	Known Limitations and Future Evaluation Work	77
6.7.1	Open Issues	77
6.7.2	Coverage Gaps	78
6.8	Summary	78
7	Conclusion and Future Work	79
7.1	Summary of Work	79
7.2	Contributions	80
7.3	Limitations	81
7.4	Future Work	82
7.4.1	ASIC Synthesis and Physical Design	82
7.4.2	Formal and Property-Based Verification	82
7.4.3	Instruction Set Extensions	83
7.4.4	Multi-Lane Scaling	83
7.4.5	Out-of-Order VPU Execution with Scoreboard	84
7.4.6	Clock Gating and Power Optimisation	84
7.5	Closing Remarks	84

List of Figures

3.1	Harvard memory architecture: IMEM connected to the IF stage (read-only), DMEM with dual scalar/VLSU ports connected to the MEM stage and vector LSU.	23
4.1	Top-level block diagram of <code>riscv_vpu_top</code> : scalar pipeline, VPU subsystem, IMEM, DMEM (dual-port), and interconnect buses.	30
4.2	Five-stage scalar pipeline datapath of <code>pipelined_vpu</code>	32
4.3	Block diagram of the <code>vproc_system_wrapper</code> VPU subsystem	35
4.4	VPU execution Finite State Machine (FSM) state diagram (nine states)	38
4.5	Execution lane block diagram of <code>vproc_processor_lane</code> : operand muxes, seven parallel functional units, result selection mux, and mask application stage.	42
4.6	ModelSim waveform of the reduction unit executing <code>vredsum.vs</code> : FSM state transitions, <code>busy/done</code> handshake, per-element <code>acc_r</code> accumulation, and VRF writeback.	46
4.7	VSE8.v masked store timing diagram	48
5.1	Firmware build flow diagram: <code>.S</code> source $\rightarrow$ <code>.o</code> (assemble) $\rightarrow$ <code>.elf</code> (link) $\rightarrow$ <code>.bin</code> (objcopy) $\rightarrow$ <code>.hex</code> (<code>elf2hex_lines.py</code>) $\rightarrow$ <code>\$readmemh</code> in <code>imem.sv</code>	56
6.1	ModelSim transcript of <code>tb_lena_gray</code> showing cycle count 34828, <code>Y[0..15]</code> output values, and “DMEM dumped” confirmation	73
6.2	Quartus Prime TimeQuest Fmax summary: 47.18 MHz fast-corner model (+2.247 ns slack) and slow-corner timing violation (−4.436 ns) for the Cyclone V target.	75

List of Tables

2.1	Comparison of RISC-V vector processor implementations	15
3.1	Arithmetic instructions (OPIVV/OPIVX/OPIVI)	18
3.2	Logical and shift instructions	18
3.3	Multiplication instructions (OPIVV / OPMVV / OPMVX)	18
3.4	Comparison (mask-generating) instructions	19
3.5	Reduction instructions (OPMVV)	19
3.6	Vector move and configuration instructions	19
3.7	Vector load/store instructions (unit-stride)	19
3.8	SEW encoding in vsegw[2:0]	20
3.9	LMUL encoding in vlmul[2:0]	21
3.10	VLMAX values for VLEN=128	21
3.11	vtype CSR register layout	22
3.12	Instruction memory specification	23
3.13	Data memory specification	24
3.14	UART MMIO register map	25
3.15	UART STAT register bit assignments	25
4.1	Top-level inter-subsystem buses and their widths	31
4.2	49-bit ctrl_bus field encoding	36
4.3	FSM output signals asserted in each state	39
4.4	UART memory-mapped register map (base address 0xFF00_0000)	50
4.5	System memory address map	52
4.6	RTL module summary	53
5.1	Software build targets and their purposes	57
5.2	UART MMIO register map	60
5.3	VPU instruction cycle counts for SEW=8, vl=16	64
6.1	Instruction regression coverage: 172/172 test cases passing	70
6.2	Representative test vectors from tb_vproc_all_instr	71
6.3	Representative BT.601 pixel test vectors — all match reference	72
6.4	VPU benchmark results (simulated cycle counts)	74
6.5	Quartus Prime synthesis results: DE10-Standard, Cyclone V 5CSXFC6D6F31C6	75
6.6	Fmax evolution through design iterations	76

List of Abbreviations

ALM	Adaptive Logic Module
ASIC	Application-Specific Integrated Circuit
CSR	Control and Status Register
DRC	Design Rule Check
FIFO	First-In First-Out
Fmax	Maximum Operating Frequency
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
ISA	Instruction Set Architecture
LMUL	Length MULtiplier
LSU	Load/Store Unit
LVS	Layout vs. Schematic
PnR	Place and Route
RISC-V	Reduced Instruction Set Computer V
RTL	Register Transfer Level
SEW	Standard Element Width
SIMD	Single Instruction Multiple Data
STA	Static Timing Analysis
TL-UL	TileLink Uncached Lightweight
UART	Universal Asynchronous Receiver-Transmitter
VLEN	Vector Register Length
VLSU	Vector Load/Store Unit
VPU	Vector Processing Unit
VRF	Vector Register File

Introduction

1.1 Motivation and Background

The past decade has witnessed an unprecedented surge in data-parallel workloads driven by machine learning inference, computer vision, digital signal processing, and scientific simulation. Modern neural-network inference passes millions of multiply-accumulate operations over tensors that may contain thousands of elements of identical type. A classical scalar processor that fetches, decodes, and executes one operation per clock cycle cannot amortise the instruction-fetch overhead across these repeated computations; it spends the majority of its energy on control flow rather than useful arithmetic. The fundamental insight that motivates vector processing is simple: if the same arithmetic operation is to be applied to a long sequence of operands, a single instruction can describe the entire sequence, and the hardware can overlap the execution of individual elements in time, in space, or both.

Vector processing architectures have a history stretching back to the Cray-1 vector supercomputer of 1976, yet they have undergone a dramatic renaissance in the era of deep learning. Modern CPU vector extensions — Intel AVX-512, ARM SVE/SVE2, and RISC-V RVV — share the same conceptual heritage: a set of wide registers holding multiple data elements and an instruction vocabulary that operates on all elements simultaneously [1]. These extensions achieve substantial performance improvements on workloads such as image convolution, matrix multiplication, and activation functions, often by factors of four to sixteen compared to scalar code on the same core, with minimal additional area overhead when the vector width is chosen judiciously.

The RISC-V open instruction set architecture has emerged as a compelling foundation for custom silicon development [2, 3]. Its permissive licensing model, its separation of mandatory base ISA from optional standard extensions, and its active ecosystem of open-source toolchains and verification infrastructure make it uniquely attractive for academic and start-up design teams that cannot afford proprietary ISA licenses. The RISC-V Vector Extension (RVV), standardised as version 1.0 in 2021 [4], defines a length-agnostic programming model that allows the same binary to run correctly on vector units ranging from 128-bit to 65536-bit register widths, simply by rerunning the `vsetvli` configuration instruction at runtime. This future-proof abstraction

is particularly valuable for [Application-Specific Integrated Circuit \(ASIC\)](#) design because the hardware can be dimensioned to the minimum area required for the target workload without invalidating software compiled for wider implementations.

The choice between [Field-Programmable Gate Array \(FPGA\)](#) prototyping and [ASIC](#) tapeout reflects a fundamental engineering tradeoff. An [FPGA](#) provides rapid iteration and in-system debugging at the cost of power efficiency, achievable clock frequency, and configurable logic resources. An [ASIC](#), once fabricated, consumes three to five times less power and operates at two to four times higher frequency for equivalent logic, because every transistor is sized and placed for a single purpose [5]. For embedded inference at the edge — smart cameras, wearable health monitors, autonomous vehicle perception nodes — the power envelope is critical, and [ASIC](#) implementation is the only viable path. Producing tapeout-ready [Register Transfer Level \(RTL\)](#) therefore has direct industrial relevance beyond the academic context of this thesis.

Writing [RTL](#) that is not merely functionally correct in simulation but also fully synthesisable, timing-clean, and free of latches and X-propagation paths is a substantially harder problem than simulation-only design. Synthesisers reject `initial` blocks, non-synthesisable system tasks, and asynchronous latches. Static timing analysis tools expose combinational paths that violate setup and hold constraints. Any path that is not properly constrained can silently cause functional failures after place-and-route. This thesis takes the position that a vector processor should be designed to these tighter standards from the outset, rather than retrofitted after the fact.

1.2 Problem Statement

Designing a standards-compliant vector processor presents multiple interacting technical challenges that make it substantially more complex than a straightforward scalar core.

ISA Complexity. The RVV Zve32x subset encompasses more than fifty distinct instruction encodings across arithmetic, logical, shift, compare, reduction, and load/store categories. Each instruction must be correctly decoded from a 32-bit encoding that shares the OP-V opcode but varies along three independent axes: `funct6` (6 bits specifying the operation), `funct3` (3 bits selecting the operand type: VV, VX, or VI), and `vm` (1 bit enabling or disabling masking). The decoder must distil this information into a compact control word that drives downstream execution units over potentially many clock cycles, because vector operations execute once per element rather than once per instruction.

Hazard Management. A combined scalar–vector system requires hazard resolution at two distinct time scales. The scalar pipeline must detect read-after-write hazards between consecutive instructions on a cycle-by-cycle basis using forwarding and load-use stalling. Concurrently, the scalar core must stall when it encounters a vector instruction while the [Vector Processing Unit](#)

(VPU) instruction queue is full, and it must stall again when a `vsetvl/vsetvli` instruction requires the VPU to complete its configuration and write the computed vector length back to a scalar register. Both stall conditions must be detected and propagated through the pipeline without corrupting any in-flight instructions.

Area and Power Tradeoffs. A multi-lane VPU with $VLEN=128$ bits and four 32-bit lanes executing simultaneously yields peak throughput but occupies proportionally more area. For an ASIC tapeout constrained by educational budget, the single-lane implementation trades peak throughput for area efficiency, completing one 32-bit element per cycle. The reduction unit is particularly sensitive: a naïve reduction tree with sixteen adders connected in a combinational chain produces a long critical path that limits F_{max} . The design must replace this with a serialised approach that achieves correct results while remaining timing-clean.

Masked Operations. RVV masking allows each vector element to be conditionally suppressed based on the contents of vector register `v0`. For compare instructions that *produce* a mask result, the implementation must accumulate per-element bits into a mask buffer and commit the result atomically at the end of the instruction, rather than writing partial results to the register file mid-execution. This requires an additional FSM state (`ST_FINAL_MASKING`) and a separate mask write buffer, both of which add control complexity without adding arithmetic throughput.

Memory Interface. A Harvard memory architecture with separate instruction and data memories simplifies timing by eliminating structural hazards on the memory bus, but requires careful arbitration when both the scalar Load/Store Unit (LSU) and the vector Vector Load/Store Unit (VLSU) wish to access data memory simultaneously. The dual-port memory solution must be synthesisable (i.e., mappable to block-RAM primitives) and must not introduce dead cycles into either the scalar or vector memory access path.

1.3 Objectives

This project has the following specific technical objectives:

1. **Implement the `rv32im_zicsr_zve32x_zv1128b` ISA.** Deliver a scalar core supporting the RV32I base, the M multiplication/division extension, and the Zicsr privileged instruction extension, integrated with a vector coprocessor that implements the full Zve32x minimum vector subset at $VLEN=128$ bits.
2. **Five-stage pipelined scalar core.** The scalar core shall implement IF, ID, EX, MEM, and WB stages with `EX→MEM→WB` data forwarding and load-use hazard stalling, achieving CPI approaching 1.0 on non-memory, non-vector code.

3. **Single-lane, multi-cycle VPU with full Zve32x coverage.** The VPU shall process one SEW-wide element per clock cycle via a single execution lane containing an adder, multiplier, shifter, logic, comparator, min/max, and reduction sub-unit. It shall support SEW=8/16/32 and LMUL=1/2/4/8.
4. **Tapeout-viable RTL.** All modules in the `/rtl/` hierarchy shall be synthesisable: no initial blocks, no `$display` in synthesisable paths, no combinational latches, synchronous active-low reset throughout, and all widths parameterised to avoid hardcoded VLEN/SEW bit slices.
5. **Comprehensive functional verification.** A directed testbench suite shall achieve 172/172 test cases passing before any design change is considered complete. Integration benchmarks shall include BT.601 RGB-to-grayscale conversion on a 128×128 image and general matrix multiplication, both verified at pixel/element level.
6. **UART-based end-to-end system test.** A UART peripheral mapped at `0xFF000000` shall enable real-data streaming from a host computer to the running system. Firmware shall receive raw image data, invoke the vector grayscale pipeline, and return the luminance channel, providing an end-to-end integration test beyond pure simulation.

1.4 Contributions

The following are the original contributions of this thesis:

1. **A complete open-source Zve32x implementation in SystemVerilog.** This work delivers the first publicly documented single-lane Zve32x implementation that explicitly targets ASIC tapeout rules, including parameterised widths, synchronous active-low reset, no latches, and no simulation-only constructs in any production RTL file.
2. **A 48-bit structured control bus architecture.** The `vproc_vdecoder` module produces a 48-bit control word that encodes all execution context — register addresses, operation flags, masking controls, reduction flags, and `vtype` — into a single bus that passes through the instruction **First-In First-Out (FIFO)** and drives all downstream units without any re-decoding. This single-decode, single-pipeline-register design minimises routing complexity and ensures synthesis tools can easily register the control path.
3. **A serial reduction unit with busy-based protocol.** Unlike published designs that employ a combinational reduction tree (which produces a $O(\log_2 N)$ -deep critical path), the `vproc_reduction` module accumulates one element per clock cycle using a single adder under FSM control. The `reduction_busy` signal allows the FSM to overlap the next instruction's decode with the final accumulation cycle, improving utilisation without increasing logic depth.
4. **A dual-port DMEM arbitration scheme for scalar–vector coexistence.** The `dmem_sync` module exposes independent scalar and VLSU ports that map to the two read/write ports of

a single block-RAM primitive, eliminating structural hazards between scalar load/store and vector load/store without an explicit arbiter.

5. **A UART TL-UL slave interface for end-to-end image streaming.** A synthesisable UART peripheral following the TileLink Uncached Lightweight ([TileLink Uncached Lightweight \(TL-UL\)](#)) protocol is integrated at the top-level address space. Firmware running on the RISC-V core can stream raw image data bytes via MMIO writes and reads, enabling end-to-end validation that no simulation-level abstraction could provide.
6. **Validated FPGA prototype on DE10-Standard achieving 47.18 MHz.** The complete design was synthesised and placed-and-routed using Intel Quartus Prime on a Cyclone V device, consuming 10 438 [Adaptive Logic Modules \(ALMs\)](#) and achieving an [Maximum Operating Frequency \(Fmax\)](#) of 47.18 MHz — a $3\times$ frequency improvement over an earlier iteration that used a combinational reduction chain, demonstrating the benefit of the serial reduction redesign.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows.

Chapter 2 provides the theoretical and contextual background required to understand the design decisions made in subsequent chapters. It surveys the RISC-V instruction set architecture, the principles of vector processing and how they differ from scalar SIMD, the RVV specification in detail, and the standard ASIC design methodology from [RTL](#) through tapeout. The chapter concludes with a comparative survey of related academic and industrial vector processor implementations, positioning this work within the broader landscape.

Chapter 3 defines the complete system specification that guided the implementation. It states the functional and non-functional requirements, provides a detailed instruction table covering all supported encodings, specifies the vector configuration model (VLEN, SEW, LMUL, VLMAX), describes the Harvard memory architecture with precise address maps, specifies the UART peripheral's register file and timing constraints, and states the performance targets against which the design is evaluated.

Chapter 4 describes the hardware architecture and [RTL](#) implementation in detail. It covers the five-stage scalar pipeline with hazard detection and forwarding, the vector decoder and its 48-bit control bus, the instruction [FIFO](#), the nine-state execution [FSM](#), the single-lane execution pipeline and each of its sub-units, the vector register file, the vector load/store unit, and the UART peripheral.

Chapter 5 describes the firmware and software infrastructure. It covers the assembly language programming model exposed to firmware writers, the build system and toolchain configuration, the BT.601 grayscale benchmark, the UART streaming protocol, and the `elf2hex` conversion pipeline that loads programs into the hardware.

Chapter 6 presents verification and evaluation results. It describes the testbench methodology, the 172-case regression suite, per-instruction pass/fail analysis, the 128×128 pixel grayscale benchmark, FPGA timing and area results, and a discussion of the frequency bottleneck and how the serial reduction redesign resolved it.

Chapter 7 summarises the contributions, discusses limitations of the current implementation, and outlines directions for future work including multi-lane scaling, floating-point support, and formal verification.

Background and Related Work

2.1 RISC-V Instruction Set Architecture

2.1.1 History and Design Philosophy

The RISC-V project began at the University of California, Berkeley in 2010 under the direction of Krste Asanović and David Patterson, motivated by the observation that every major instruction set architecture either carried decades of backwards- compatibility baggage (x86, ARM) or was encumbered by licensing terms that inhibited academic and open-source development [3]. The design team deliberately started from a clean slate, taking the well-understood principles of reduced instruction set computing and applying them with the benefit of thirty years of hindsight.

The central tenets of the RISC-V philosophy are modularity, simplicity, and openness. Rather than defining a single monolithic instruction set, RISC-V specifies a small mandatory base ISA (RV32I or RV64I) and a collection of standard extensions that can be combined selectively. A chip targeting IoT sensor fusion needs only RV32I and perhaps the C (compressed) extension; a high-performance server processor adds the M, A, F, D, and V extensions. The letter “G” denotes the general-purpose combination *IMAFD*. This modularity allows silicon area to be allocated precisely to the features that a given application domain demands.

The open-source nature of RISC-V distinguishes it from all commercial alternatives. The specifications are published under Creative Commons licensing, the reference implementations (Rocket, BOOM, CVA6) are available under BSD or Apache licenses, and a rich ecosystem of open-source tools — GCC, LLVM, Spike ISA simulator, Verilator, and ModelSim flows — eliminates the toolchain cost that historically made ASIC design prohibitive for academic groups.

2.1.2 Base ISA: RV32I

RV32I defines thirty-two 32-bit general-purpose integer registers (x_0 – x_{31} , where x_0 is hardwired to zero), a 32-bit program counter, and a load/store memory architecture in which all arithmetic operates on registers and memory is accessed exclusively through explicit load and store instructions [2]. The instruction encoding is fixed at 32 bits (the C extension provides

optional 16-bit compressed forms), and all instructions are aligned on 4-byte boundaries in the standard configuration.

The RV32I instruction formats — R, I, S, B, U, and J — are designed to keep the source and destination register fields at fixed bit positions (bits [11:7] for `rd`, [19:15] for `rs1`, [24:20] for `rs2`) regardless of format, reducing decoder complexity. The opcode occupies bits [6:0] and the `funct3/funct6/funct7` fields further disambiguate within an opcode class.

The base integer instruction set covers arithmetic (ADD, SUB, ADDI), logical (AND, OR, XOR and their immediate forms), shift (SLL, SRL, SRA), comparison (SLT, SLTU), branch (BEQ, BNE, BLT, BGE, BLTU, BGEU), unconditional jump (JAL, JALR), and memory access (LW, LH, LB, LHU, LBU, SW, SH, SB).

2.1.3 M Extension: Integer Multiplication and Division

The M extension adds hardware multiply and divide instructions to RV32I [2]. MUL produces the lower 32 bits of the product; MULH, MULHU, and MULHSU produce the upper 32 bits for signed×signed, unsigned×unsigned, and signed×unsigned cases respectively. DIV, DIVU, REM, and REMU perform integer division and remainder with signed and unsigned variants. Without the M extension, multiplication must be emulated in software using shift-and-add sequences, which is impractical for the matrix multiplication and convolution kernels that motivate this design.

2.1.4 Privileged Specification and CSRs

The Zicsr extension defines a uniform mechanism for accessing [Control and Status Registers \(CSRs\)](#) using the CSRRW, CSRRS, CSRRC, and their immediate variants (CSRRWI, CSRRSI, CSRRCI) [2]. CSR addresses occupy a 12-bit field in the instruction encoding (`inst[31:20]`), giving a 4096-entry address space partitioned by privilege level and read/write access. The vector extension uses this mechanism to expose three user-level CSRs: `v1` (current vector length, address 0xC20), `vtype` (vector type register encoding SEW, LMUL, and tail/mask policy, address 0xC21), and `vlenb` (vector register length in bytes, read-only constant, address 0xC22).

2.2 Vector Processing Fundamentals

2.2.1 Flynn’s Taxonomy and Parallelism Classes

Flynn’s taxonomy [6] classifies computer architectures by the number of instruction streams and data streams they operate on simultaneously: Single Instruction Single Data (SISD) describes a classical scalar processor; Multiple Instruction Multiple Data (MIMD) describes a multi-core

system; Single Instruction Multiple Data (SIMD) describes the class that encompasses both vector processors and modern packed-SIMD extensions.

Within the [Single Instruction Multiple Data \(SIMD\)](#) class, a useful distinction separates *true vector* processors from *packed SIMD* designs. In packed SIMD (MMX, SSE, AVX, NEON), the vector register width is fixed in the [Instruction Set Architecture \(ISA\)](#) and the programmer explicitly handles loop tails when the data set length is not a multiple of the fixed vector width. In a true vector processor, the [ISA](#) is parameterised by a hardware-configured vector length that the application queries at runtime using a configuration instruction; the hardware handles tail elements automatically. RISC-V RVV follows the true vector model.

2.2.2 Key Vector Concepts

Vector Length and VLEN. [Vector Register Length \(VLEN\)](#) is the number of bits in each vector register. For a given VLEN and SEW (the number of bits per element), the number of elements per register is $VLEN/SEW$. This thesis fixes $VLEN=128$ bits, so a register holds sixteen 8-bit elements, eight 16-bit elements, or four 32-bit elements when $LMUL=1$.

Standard Element Width (SEW). [Standard Element Width \(SEW\)](#) specifies the data type width for a vector instruction. The RVV specification encodes SEW in a three-bit field `vsew[2:0]` within the `vtype` CSR. Zve32x mandates support for $SEW=8, 16$, and 32 bits.

Length Multiplier (LMUL). [Length Multiplier \(LMUL\)](#) groups consecutive vector registers into a single logical register, scaling the effective vector length by the multiplier. $LMUL=1$ is the base case; $LMUL=2$ pairs two registers, doubling the number of elements that a single instruction operates on. $LMUL=4$ and $LMUL=8$ continue the progression. Fractional LMUL values ($LMUL=1/2, 1/4, 1/8$) use only a fraction of a register and are defined in the full V extension but not required by Zve32x. The maximum vector length for a given SEW and LMUL is:

$$VLMAX = \left\lfloor \frac{LMUL \times VLEN}{SEW} \right\rfloor \quad (2.1)$$

Masking. RVV provides a per-element masking mechanism. When the `vm` bit in an instruction is 0, each element operation is conditioned on the corresponding bit of vector register `v0`. Elements whose mask bit is 0 are either left unchanged (mask-undisturbed policy) or written with all-ones (mask-agnostic policy), depending on the `vma` bit in `vtype`. Masking enables vectorised control flow: a conditional operation on a subset of elements can be expressed as a comparison followed by a masked arithmetic instruction without branching.

Tail Behaviour. When `vl` is set to a value less than $VLMAX$, the elements from index `vl` to $VLMAX-1$ are “tail” elements. The tail-agnostic (`ta`) bit in `vtype` controls whether tail elements

are left unchanged or overwritten with all-ones. This thesis implements tail-agnostic behaviour, writing zero to tail positions in the register file.

2.2.3 Performance Advantages of Vector Processing

The performance advantage of vector processing over scalar processing for data-parallel workloads stems from three sources. First, instruction-fetch overhead is amortised: a single `vadd.vv` instruction replaces N scalar `add` instructions, reducing instruction cache pressure proportionally. Second, branch prediction overhead is eliminated because the innermost loop over elements is implicit in the vector length counter inside the hardware. Third, the hardware can pipeline element operations in a deeper pipeline than a scalar functional unit, because the latency can be hidden behind the processing of subsequent elements within the same vector instruction.

For a concrete estimate, a BT.601 grayscale conversion applied to a 128×128 image requires 16384 multiply-accumulate operations per colour channel. A scalar RV32I core at 50 MHz completes this in approximately 98304 cycles (6 instructions per pixel). A vector core with $VLEN=128$ and $SEW=8$ processes 16 elements per instruction; the same workload requires only 1024 `vmulhu.vx + vredsum.vs` passes, a $16 \times$ reduction in instruction count assuming the bottleneck is arithmetic throughput.

2.3 RISC-V Vector Extension (RVV)

2.3.1 Overview and Design Goals

The RISC-V Vector Extension, version 1.0, was ratified by RISC-V International in 2021 [4]. Its primary design goal is to provide a single binary-compatible vector programming model across a wide range of hardware implementations, from microcontrollers with $VLEN=64$ to server processors with $VLEN=4096$. The key architectural mechanism enabling this is the `vsetvl/vsetvli/vsetivli` instruction family, which configures the hardware at runtime and returns the actual vector length that the hardware can process in one instruction, allowing software to loop without prior knowledge of the hardware's capabilities.

2.3.2 Vector CSRs

Three CSRs define the current vector execution context:

- **vtype** (address `0xC21`): encodes `vill` (illegal vtype flag), `vma` (mask-agnostic), `vta` (tail-agnostic), `vsew[2:0]`, and `vlmul[2:0]`.
- **vl** (address `0xC20`): holds the current vector length as set by the most recent `vsetvl*` instruction.

- **vlenb** (address `0xC22`): a read-only constant equal to $VLEN/8$, i.e., the number of bytes in one vector register. For $VLEN=128$, $vlenb=16$.

2.3.3 Vector Register File

RVV defines 32 vector registers, $v0-v31$, each $VLEN$ bits wide. Register $v0$ has the special role of the mask register when $vm=0$. With $LMUL>1$, groups of registers are treated as a single logical register; for example, $LMUL=4$ means that $v0, v4, v8, \dots$ are the accessible register group addresses, and each group contains four physical $VLEN$ -bit registers holding $4 \times VLMAX$ elements.

2.3.4 Instruction Categories

RVV instructions are categorised by their operand encoding:

- **OPIVV** ($funct3=000$): vector–vector operations (both operands from the vector register file).
- **OPIVX** ($funct3=100$): vector–scalar operations (one operand from a scalar integer register).
- **OPIVI** ($funct3=011$): vector–immediate operations (one operand is a 5-bit sign-extended immediate).
- **OPMVV** ($funct3=010$): multiply-add and integer reduction vector–vector operations.
- **OPMVX** ($funct3=110$): multiply-add vector–scalar operations.
- **OPFVV/OPFVF**: floating-point categories (not implemented in Zve32x).
- **Load/Store**: unit-stride, strided, and indexed variants (Zve32x requires only unit-stride).

2.3.5 Zve32x Subset

The Zve32x profile is the minimum embedded vector extension targeting integer-only workloads at 32-bit maximum element width [4]. It requires: $VLEN \geq 32$, SEW support for 8, 16, and 32 bits, all integer arithmetic and logical operations with VV/VX/VI encodings, integer comparison and mask operations, reduction operations ($vredsum, vredmax, vredmin$ and unsigned variants), and unit-stride vector load/store ($vle8.v, vle16.v, vle32.v$, and matching $vse*$). It explicitly does not require floating-point operations, strided or indexed memory accesses, or $VLEN$ exceeding 32 bits. This makes it the natural profile for a first-generation tapeout targeting embedded workloads.

2.4 ASIC Design Methodology

2.4.1 RTL Design Phase

The [RTL](#) design phase produces synthesisable SystemVerilog (or VHDL) that describes the logical behaviour of the circuit at the level of register transfers and combinational functions [6, 5]. Critical rules that distinguish synthesisable from simulation-only RTL include:

- **No initial blocks.** Initial blocks are interpreted by simulators to set power-on state but are not supported by synthesis tools. All registers must be initialised through reset logic.
- **Synchronous, active-low reset.** A single active-low `rst_n` signal, sampled on the rising clock edge, is the industry standard for ASIC reset. Asynchronous resets complicate static timing analysis because they can produce glitches during assertion and deassertion.
- **No latches.** An `always_comb` block that does not assign all outputs on all code paths infers a level-sensitive latch. Latches cause timing violations and are eliminated by ensuring every `always_comb` block has a complete set of default assignments before any conditional logic.
- **No simulation tasks in synthesis paths.** `$display`, `$time`, `$monitor`, and similar system tasks are simulation-only; they must be confined to testbenches.
- **Parameterised widths.** Hardcoding bit widths to specific VLEN or SEW values makes re-targeting to a different configuration require manual search-and-replace operations that are error-prone. Using `parameter` and `localparam` with derived widths ensures that changing one constant propagates correctly.

2.4.2 Logic Synthesis

Synthesis translates the RTL description into a gate-level netlist using a target technology library (standard cell library for ASIC or LUT library for FPGA). The synthesiser performs several optimisation passes: technology mapping selects cells from the library; retiming may move registers across combinational logic to balance stage depths; and area/power optimisation reduces the gate count while respecting timing constraints [5]. The output is a netlist of interconnected standard cells and a delay annotated timing model.

2.4.3 Static Timing Analysis

[Static Timing Analysis \(STA\)](#) verifies that the design meets its clock frequency specification without running gate-level simulation. It computes the worst-case propagation delay along every combinational path from one register's output to the next register's input, accounting for process, voltage, and temperature (PVT) corners. A path that arrives at a flip-flop's data input too close to the rising clock edge (setup violation) or too quickly after a previous clock edge (hold violation)

will cause a functional failure in silicon. For the designs in this thesis, the FPGA timing analyser (Quartus Prime TimeQuest) plays the role of STA, and the Fmax reported is the reciprocal of the critical-path delay under slow-corner conditions.

2.4.4 Place and Route

Place and Route (PnR) takes the synthesised netlist and assigns each cell a physical location on the chip (placement) and then connects them using metal routing (routing). In a full ASIC flow, this step is performed using tools such as Cadence Innovus or Synopsys ICC2. The routed design produces a GDS-II file, the industry standard for mask fabrication. For FPGA, Quartus Prime Fitter performs placement onto the array of adaptive logic modules and routes through the programmable interconnect.

2.4.5 Physical Verification: DRC and LVS

Design Rule Check (DRC) checks that all geometries in the GDS comply with the foundry's manufacturing rules: minimum feature sizes, spacing between metal layers, via densities, and antenna rules. **Layout vs. Schematic (LVS)** compares the extracted netlist from the GDS (derived by recognising transistor structures from the layout polygons) against the schematic netlist from synthesis, verifying that the two are topologically equivalent. Both checks are mandatory before any design is submitted for fabrication.

2.5 Related Work

2.5.1 Ara: Scalable RVV Vector Processor in 22nm FD-SOI

Ara is a scalable, application-class RVV vector processor developed at ETH Zürich and tapeout in TSMC 22nm FD-SOI technology [7]. It is coupled to the CVA6 (formerly Ariane) 64-bit out-of-order RISC-V scalar core and supports the full RISC-V V extension including floating-point. The design is parameterised by the number of lanes, with published results for two-lane, four-lane, and eight-lane configurations. At four lanes, Ara achieves 1 GHz+ operating frequency and delivers 12.5 GFLOP/s on double-precision floating-point reductions.

Ara's lane architecture assigns a fixed set of vector registers to each lane, enabling all lanes to process their assigned element slice in parallel. The register file in each lane stores VLEN/NLANE bits. An operand requester unit manages read-after-write hazards between vector instructions through a scoreboard. The memory system uses a banked scratchpad SRAM with one bank per lane to avoid memory bottlenecks.

Ara represents the current state of the art for high-performance RVV implementations but is significantly more complex than a first-generation tapeout target: its lane count parameterisation,

out-of-order scalar core coupling, floating-point support, and full V extension compliance require a design team of several engineers and a budget commensurate with industrial silicon development.

2.5.2 CVA6 / Ariane

CVA6 is the scalar core paired with Ara [8]. It is a six-stage, in-order, single-issue 64-bit RISC-V core targeting application-class embedded Linux workloads. Its contributions most relevant to this thesis are the demonstration that a small team can produce a tapeout-quality RISC-V implementation in SystemVerilog with thorough verification, and its published FD-SOI results showing 1.7 GHz at 1.1 V. The CVA6 core also serves as the reference for the scalar–vector dispatch interface that RVV processors use: the scalar core stalls when the vector unit is busy, forwarding the vector instruction and associated scalar register values directly to the coprocessor. This thesis adopts the same dispatch paradigm for the scalar–VPU interface.

2.5.3 Vitruvius+

Vitruvius+ is an area-efficient RVV vector coprocessor for high-performance computing developed at the Barcelona Supercomputing Centre [9]. It emphasises decoupled memory access by providing a large number of outstanding memory requests, hiding DRAM latency for streaming kernels. Vitruvius+ integrates with RISC-V scalar cores via the RVXIF (RISC-V External Interface) extension, an emerging standard for coprocessor coupling that provides a more structured handshake than ad-hoc signal naming. Its vector register file is distributed across multiple SRAM macros with careful power gating to reduce idle-state power.

2.5.4 Hwacha

Hwacha is a decoupled vector accelerator developed at UC Berkeley as a predecessor to RVV, using a custom ISA rather than the standardised RVV encoding [10]. It introduced several concepts that influenced the RVV design, including the separation of vector configuration (the `vsetvl` idiom) from execution, the decoupled memory access architecture (where vector loads issue independently of computation), and the notion of a vector fetch unit that sequences element-wise operations from a single instruction. Hwacha was fabricated in TSMC 28nm and validated silicon results showed correct execution of linear algebra kernels.

2.5.5 Comparative Analysis

Table 2.1 summarises the key characteristics of the related designs compared to this work.

Table 2.1. Comparison of RISC-V vector processor implementations

Design	ISA	VLEN	Lanes	Tech	Target
Ara [7]	RVV 1.0 (full)	512–4096	2–8	22nm FD-SOI	HPC/Server
CVA6 [8]	RV64IMAFDC	N/A	N/A	22nm FD-SOI	App-class
Vitruvius+ [9]	RVV 1.0	512	8	7nm	HPC
Hwacha [10]	Custom (pre-RVV)	2048	8	28nm	HPC
This work	Zve32x	128	1	FPGA (Cyclone V)	Embedded

The fundamental distinction between this work and the related designs is the design target. All of Ara, Vitruvius+, and Hwacha target high-performance computing workloads in advanced technology nodes (7–28nm). They prioritise throughput, utilising multiple parallel lanes, large VLENs, and sophisticated out-of-order scalar cores. This thesis targets the opposite point in the design space: a minimum-area, tapeout-viable implementation of the Zve32x subset intended for embedded edge inference. The single-lane architecture, 128-bit VLEN, and 5-stage in-order scalar core reflect deliberate area minimisation for a first-generation student tapeout.

The closest academic parallel is the open-source work on small embedded RVV processors, but to the authors’ knowledge, no published work explicitly documents a complete single-lane Zve32x implementation with end-to-end functional verification on image processing benchmarks in publicly available SystemVerilog RTL, making this a distinct contribution to the open-source RISC-V hardware ecosystem.

2.6 Summary

This chapter has established the technical foundations required to understand the design decisions in subsequent chapters. The RISC-V ISA provides a modular, open framework; its RV32IM base gives the scalar core its programming model, and the Zicsr extension provides the mechanism through which the vector unit exposes its configuration state. Vector processing fundamentally improves data-parallel performance by amortising instruction-fetch overhead across many elements, and the RVV Zve32x subset defines a minimum yet complete integer vector profile suitable for embedded inference. The ASIC design methodology sets strict synthesizability rules that are enforced throughout the RTL. Finally, the related work survey shows that while high-performance many-lane implementations exist, the single-lane embedded Zve32x target of this thesis occupies a distinct and underserved point in the design space.

System Specification

3.1 System Requirements

3.1.1 Functional Requirements

The system shall implement the `rv32im_zicsr_zve32x_zvl128b` instruction set architecture. The following functional requirements are derived from the RVV specification [4] and the project objectives stated in Chapter 1.

- FR-1. The scalar core shall execute all RV32I base instructions and all RV32M multiply/divide instructions.
- FR-2. The scalar core shall support CSR access instructions (`CSRRW`, `CSRRS`, `CSRRC`, and their immediate variants) to read the vector length, vector type, and vector length-in-bytes registers.
- FR-3. The **VPU** shall implement all Zve32x integer arithmetic instructions: `vadd`, `vsub`, `vrsb`, `vmul`, `vmulh`, `vmulhu`, `vmulhsu`, and their widening variants.
- FR-4. The **VPU** shall implement logical instructions: `vand`, `vor`, `vxor` in VV, VX, and VI encoding variants.
- FR-5. The **VPU** shall implement shift instructions: `vsll`, `vsrl`, `vsra` in VV, VX, and VI variants.
- FR-6. The **VPU** shall implement integer comparison and mask-generating instructions: `vmseq`, `vmsne`, `vmslt`, `vmsltu`, `vmsle`, `vmsleu`, `msgt`, `msgtu`.
- FR-7. The **VPU** shall implement integer reduction instructions: `vredsum`, `vredmax`, `vredmin`, `vredmaxu`, `vredminu`.
- FR-8. The **VPU** shall implement unit-stride vector load and store instructions: `vle8.v`, `vle16.v`, `vle32.v`, `vse8.v`, `vse16.v`, `vse32.v`.
- FR-9. The **VPU** shall implement integer min/max instructions: `vmin`, `vmax`, `vminu`, `vmaxu`.
- FR-10. The **VPU** shall implement `vmv.v.x` (broadcast scalar to all elements) and `vmv.x.s` (move element 0 to scalar register).
- FR-11. The **VPU** shall implement carry-in arithmetic: `vadc`, `vmadc`.
- FR-12. The **VPU** shall implement vector configuration instructions: `vsetvli`, `vsetvl`,

`vsetivli`.

- FR-13. The system shall include a UART peripheral at memory address `0xFF000000` supporting 8-N-1 framing at 115200 baud, accessible through MMIO load/store instructions.
- FR-14. The system shall support Harvard memory architecture: a read-only 8 KB instruction memory at `0x00000000` and a read/write 64 KB data memory at `0x00000000` in the data address space.

3.1.2 Non-Functional Requirements

- NF-1. **Synthesizability.** All RTL in the `/rtl/` directory shall be synthesisable by Quartus Prime and by standard ASIC synthesis tools (Yosys, Synopsys DC). No `initial` blocks, no `$display/$time` tasks, no combinational latches, and no delays (`#N`) shall appear in any synthesisable file.
- NF-2. **Synchronous active-low reset.** All registers shall be initialised through a synchronous active-low `rst_n` signal. No asynchronous reset paths shall exist in the production RTL (the FPGA-specific `d_ff.sv` wrapper is excluded from this requirement).
- NF-3. **Parameterised widths.** All bus widths that depend on `VLEN`, `SEW`, or `LMUL` shall be expressed using `parameter` or `localparam` declarations. No hardcoded constants that would require manual editing to retarget the design.
- NF-4. **Target clock frequency.** The design shall target a minimum $F_{\max}$ of 50 MHz on the Intel Cyclone V device (DE10-Standard) [11] at typical process, voltage, and temperature conditions, enabling real-time processing of at least one image row per millisecond at `VLEN=128`.
- NF-5. **Verification gate.** A regression testbench of 172 directed tests shall achieve 100% pass rate before any RTL change is considered complete.
- NF-6. **Code style.** All RTL shall use `logic` types (not `wire/ reg` distinctions), `always_ff/always_comb` blocks (not `always @(*)`), and `typedef enum` for FSM state types.

3.2 ISA Specification

3.2.1 Instruction Encoding Conventions

All RVV instructions use the OP-V opcode (`7'b101_0111`). The `funct3` field selects the operand encoding class (VV, VX, VI, etc.). The `funct6` field selects the specific operation within the class. The `vm` bit (`inst[25]`) controls masking: `vm=1` means unmasked (all elements active), `vm=0` means masked by `v0`.

3.2.2 Supported Instructions by Category

Tables 3.1–3.7 list all supported instructions with their opcode, funct3, funct6, and operand type.

Table 3.1. Arithmetic instructions (OPIVV/OPIVX/OPIVI)

Instruction	funct6	funct3 (VV)	funct3 (VX)	funct3 (VI)	Notes
vadd.vv/vx/vi	000000	000	100	011	Signed/unsigned add
vsub.vv/vx	000010	000	100	—	Subtract
vrsub.vx/vi	000011	—	100	011	Reverse subtract
vmin.vv/vx	000101	000	100	—	Signed minimum
vminu.vv/vx	000100	000	100	—	Unsigned minimum
vmax.vv/vx	000111	000	100	—	Signed maximum
vmaxu.vv/vx	000110	000	100	—	Unsigned maximum
vadc.vvm/vxm	010000	000	100	—	Add with carry (vm=0)
vadc.vvm/vxm	010001	000	100	—	Mask carry out

Table 3.2. Logical and shift instructions

Instruction	funct6	funct3 (VV)	funct3 (VX/VI)	Notes
vand.vv/vx/vi	001001	000	100/011	Bitwise AND
vor.vv/vx/vi	001010	000	100/011	Bitwise OR
vxor.vv/vx/vi	001011	000	100/011	Bitwise XOR
vsll.vv/vx/vi	010101	000	100/011	Logical left shift
vsrl.vv/vx/vi	010000	000	100/011	Logical right shift
vsra.vv/vx/vi	010010	000	100/011	Arithmetic right shift

Table 3.3. Multiplication instructions (OPIVV / OPMVV / OPMVX)

Instruction	funct6	funct3 (VV)	funct3 (VX)	Notes
vmul.vv/vx	100101	010	110	Low 32 bits of product
vmulh.vv/vx	100111	010	110	Signed×signed high
vmulhu.vv/vx	100100	010	110	Unsigned×unsigned high
vmulhsu.vv/vx	100110	010	110	Signed×unsigned high
vwmul.vv/vx	111011	010	110	Widening signed×signed
vwmulu.vv/vx	111000	010	110	Widening unsigned×unsigned
vwmulsu.vv/vx	111010	010	110	Widening signed×unsigned

Table 3.4. Comparison (mask-generating) instructions

Instruction	funct6	funct3 (VV)	funct3 (VX/VI)	Result
vmseq.vv/vx/vi	011000	000	100/011	$vd[i] = (vs2[i] == vs1[i])$
vmsne.vv/vx/vi	011001	000	100/011	$vd[i] = (vs2[i] != vs1[i])$
vmsltu.vv/vx	011010	000	100	Unsigned less-than
vmslt.vv/vx	011011	000	100	Signed less-than
vmsleu.vv/vx/vi	011100	000	100/011	Unsigned $\leq$
vmsle.vv/vx/vi	011101	000	100/011	Signed $\leq$
msgtu.vx/vi	011110	—	100/011	Unsigned $>$
msgt.vx/vi	011111	—	100/011	Signed $>$

Table 3.5. Reduction instructions (OPMVV)

Instruction	funct6	funct3	Operation
vredsum.vs	000000	010	Sum of all elements + $vs1[0]$
vredmax.vs	000001	010	Signed maximum + $vs1[0]$
vredmaxu.vs	000010	010	Unsigned maximum + $vs1[0]$
vredmin.vs	000011	010	Signed minimum + $vs1[0]$
vredminu.vs	000100	010	Unsigned minimum + $vs1[0]$

Table 3.6. Vector move and configuration instructions

Instruction	Encoding	funct3	Operation
vmv.v.x	funct6=010111, VX	100	Broadcast $rs1$ to all elements
vmv.x.s	funct6=010000, VV	010	Copy element 0 to scalar rd
vsetvli	$inst[31]=0$	111	Set vtype from zimm, vl from $rs1$
vsetivli	$inst[31:30]=11$	111	Set vtype from zimm, AVL from uimm5
vsetvl	$inst[31]=1, [30]=0$	111	Set vtype from $rs2$, vl from $rs1$

Table 3.7. Vector load/store instructions (unit-stride)

Instruction	Opcode	mew/width	Mask	Operation
vle8.v	0000111	000	$vm=0/1$	Unit-stride load, SEW=8
vle16.v	0000111	101	$vm=0/1$	Unit-stride load, SEW=16
vle32.v	0000111	110	$vm=0/1$	Unit-stride load, SEW=32
vse8.v	0100111	000	$vm=0/1$	Unit-stride store, SEW=8
vse16.v	0100111	101	$vm=0/1$	Unit-stride store, SEW=16
vse32.v	0100111	110	$vm=0/1$	Unit-stride store, SEW=32

3.3 Vector Configuration

3.3.1 VLEN and Register Geometry

This design fixes VLEN=128 bits. Each of the 32 vector registers holds 128 bits. The physical register file is implemented as four sets of 32-entry $\times$ 32-bit memories (one per 32-bit lane), which together represent the 128-bit wide register at each address. This organisation allows a single 32-bit element to be read or written per clock cycle in the single-lane configuration.

3.3.2 SEW Encoding

The standard element width is encoded in the three-bit `vsew[2:0]` field of the `vtype` CSR as shown in Table 3.8.

Table 3.8. SEW encoding in `vsew[2:0]`

<code>vsew[2:0]</code>	SEW (bits)	Elements per 128-bit register (LMUL=1)
000	8	16
001	16	8
010	32	4
011–111	Reserved	—

Zve32x requires SEW support for 8, 16, and 32 bits. SEW=64 and above are not required and are not implemented.

3.3.3 LMUL Encoding

The length multiplier is encoded in the three-bit `vlmul[2:0]` field as shown in Table 3.9. Fractional LMUL values (LMUL<1) are defined in the full V extension but not required by Zve32x; they are decoded but map to LMUL=1 behaviour in this implementation.

Table 3.9. LMUL encoding in vlmul[2:0]

vlmul[2:0]	LMUL value	Support
000	1	Full
001	2	Full
010	4	Full
011	8	Full
101	1/2	Treated as LMUL=1
110	1/4	Treated as LMUL=1
111	1/8	Treated as LMUL=1
100	Reserved	—

3.3.4 VLMAX Formula

The maximum vector length for a given configuration is:

$$\text{VLMAX} = \left\lfloor \frac{\text{LMUL} \times \text{VLEN}}{\text{SEW}} \right\rfloor \quad (3.1)$$

For VLEN=128 and the supported (SEW, LMUL) combinations, the VLMAX values are as shown in Table 3.10.

Table 3.10. VLMAX values for VLEN=128

SEW	LMUL=1	LMUL=2	LMUL=4	LMUL=8
8	16	32	64	128
16	8	16	32	64
32	4	8	16	32

3.3.5 vtype Register Layout

The `vtype` register is a 32-bit read-only CSR (address 0xC21) with the layout shown in Table 3.11. The hardware stores only the lower 8 bits internally (the encoded `vtype_enc` byte) and zero-extends it to 32 bits for CSR reads.

Table 3.11. vtype CSR register layout

Bits	Field	Description
[31]	vill	1 = illegal vtype encoding
[30:8]	—	Reserved, read as zero
[7]	vma	Mask-agnostic (1 = overwrite with 1s)
[6]	vta	Tail-agnostic (1 = overwrite with 1s)
[5:3]	vsew[2:0]	Standard element width encoding
[2:0]	vlmul[2:0]	Length multiplier encoding

3.4 Memory Architecture

3.4.1 Harvard Architecture Overview

The system uses a Harvard memory architecture in which the instruction memory and the data memory occupy separate address spaces connected to the processor through independent buses. This eliminates structural hazards on the memory bus when the IF stage is fetching an instruction while the MEM stage is accessing data: both can proceed simultaneously without arbitration.

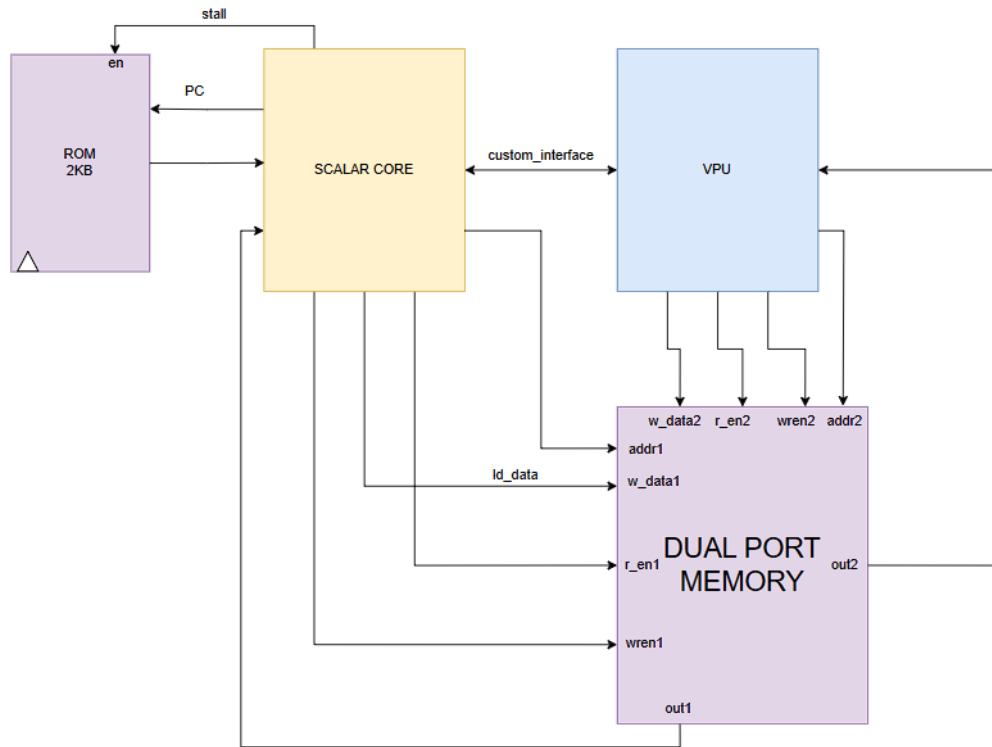


Figure 3.1. Harvard memory architecture: IMEM connected to the IF stage (read-only), DMEM with dual scalar/VLSU ports connected to the MEM stage and vector LSU.

3.4.2 Instruction Memory (IMEM)

The instruction memory is a synchronous single-port read-only memory with the parameters shown in Table 3.12.

Table 3.12. Instruction memory specification

Parameter	Value
Base address	0x00000000
Size	8 KB (2048 × 32-bit words)
Access width	32 bits
Latency	1 cycle (registered output)
Contents	.text and .rodata sections only
Initialisation	\$readmemh from imem.hex at simulation
Write access	None (read-only in hardware)

The `imem.hex` file is generated from the compiled firmware ELF by the `elf2hex_lines.py` script, which converts the raw binary into one 32-bit word per

line in hexadecimal format. The linker script (`sw/link.ld`) places `.text` at `0x00000000` and limits the total code size to 8 KB.

3.4.3 Data Memory (DMEM)

The data memory is a synchronous dual-port block RAM with the parameters shown in Table 3.13. Port A services scalar load/store requests from the scalar pipeline's MEM stage. Port B services vector load/store requests from the VLSU. Both ports can operate simultaneously, enabling the scalar and vector pipelines to access memory in the same clock cycle without mutual stalling.

Table 3.13. Data memory specification

Parameter	Port A (Scalar)	Port B (VLSU)
Base address	0x00000000	0x00000000
Size	64 KB (16384 × 32-bit words)	
Access width	32 bits (byte-enable)	32 bits (byte-enable)
Latency	1 cycle	1 cycle
Write support	Yes	Yes (vector stores)
Byte enable	4-bit mask	4-bit mask

The VLSU uses `addr[15:2]` as the word index into the 64KB DMEM, consistent with word-addressed access. Byte-enable signals allow sub-word writes for 8-bit and 16-bit vector element widths.

3.4.4 Address Decode

The scalar pipeline's LSU decodes memory addresses as follows:

- `addr[31:8] == 24'h000000`: DMEM access (routed to Port A of DMEM).
- `addr[31:8] == 24'hFF0000`: UART MMIO access (routed to UART TL-UL slave).
- All other addresses: undefined behaviour (no response, reads return zero).

The VLSU always accesses DMEM via Port B regardless of the address, since vector data is guaranteed by the firmware to reside in the DMEM range.

3.5 UART Interface Specification

3.5.1 Physical Framing

The UART interface uses the 8-N-1 framing format: one start bit (logic 0), eight data bits (LSB first), no parity bit, and one stop bit (logic 1). The baud rate is configurable via RTL parameters but defaults to 115200 baud for production firmware.

At 115200 baud, the bit period is $1/115200 \approx 8.68 \mu\text{s}$. The UART baud clock is derived by dividing the system clock by a baud divisor:

$$\text{baud_div} = \left\lceil \frac{f_{\text{clk}}}{\text{baud_rate}} \right\rceil \quad (3.2)$$

For the simulation default of $f_{\text{clk}} = 14.7456 \text{ MHz}$, $\text{baud_div}=128$, giving 128 clock cycles per bit. For the FPGA default of $f_{\text{clk}} = 50 \text{ MHz}$, $\text{baud_div}=434$.

3.5.2 MMIO Register Map

The UART is mapped as a TileLink Uncached Lightweight (TL-UL) slave at base address $0\text{xFF}000000$. Table 3.14 shows the register map.

Table 3.14. UART MMIO register map

Offset	Register	Access	Description
+0x00	CTRL	RW	Control register (reserved for future use)
+0x04	STAT	RO	Status register
+0x08	TXDAT	WO	Transmit data register
+0x0C	RXDAT	RO	Receive data register

STAT Register (offset +0x04)

Table 3.15. UART STAT register bit assignments

Bit	Field	Description
[0]	tx_ready	1 = TX shift register is empty, ready for new byte
[1]	rx_valid	1 = RX FIFO contains at least one byte
[31:2]	—	Reserved, read as zero

TXDAT Register (offset +0x08)

Writing a byte to this register loads it into the transmit shift register. Writes are ignored if `tx_ready` is 0. Firmware must poll `tx_ready` in the STAT register before each write to avoid dropping characters.

RXDAT Register (offset +0x0C)

Reading this register returns the oldest byte in the receive FIFO in bits [7:0]. Bits [31:8] read as zero. If `rx_valid` is 0, the read returns an undefined value. Firmware must poll `rx_valid` in the STAT register before each read.

3.5.3 Firmware Programming Model

A typical firmware receive-byte function polls the STAT register until `rx_valid` is set, then reads RXDAT:

```

1  # a0 = UART base address (0xFF000000)
2  # Returns received byte in a0
3  uart_getchar:
4      li    t0, 0xFF000000    # UART base
5  .wait_rx:
6      lw    t1, 4(t0)         # read STAT
7      andi  t1, t1, 2         # test rx_valid bit
8      beqz  t1, .wait_rx      # spin until byte available
9      lw    a0, 12(t0)        # read RXDAT
10     andi  a0, a0, 0xFF      # mask to 8 bits
11     ret

```

Listing 3.1. UART receive byte (polling)

```

1  # a0 = byte to transmit
2  # Uses t0--t1 as temporaries
3  uart_putchar:
4      li    t0, 0xFF000000    # UART base
5  .wait_tx:
6      lw    t1, 4(t0)         # read STAT
7      andi  t1, t1, 1         # test tx_ready bit
8      beqz  t1, .wait_tx      # spin until TX ready
9      sw    a0, 8(t0)         # write TXDAT
10     ret

```

Listing 3.2. UART transmit byte (polling)

3.5.4 Timing Constraints

The UART's TL-UL slave interface is registered: the channel-D response to a channel-A request is registered at the output, introducing a one-cycle read latency. This matches the one-cycle latency of the DMEM, ensuring that the scalar pipeline's load path has the same timing whether accessing DMEM or the UART.

For synthesis, the UART datapath must meet setup timing at the target clock frequency. The baud divisor counter is a simple 16-bit counter; its path from counter register to the baud-clock enable is not on the critical path. The TL-UL address comparator (checking `addr[31:8] == 24'hFF0000`) adds one level of logic to the address decode path, which is accounted for in the FPGA timing analysis.

3.6 Performance Requirements

3.6.1 Target Clock Frequency

The design shall achieve a minimum $F_{\max}$ of 50 MHz on the Intel Cyclone V (5CSXFC6D6F31C8) device used on the DE10-Standard board [11]. This target is chosen to enable real-time processing of a 640×480 video frame at approximately 2 frames per second with the single-lane architecture, providing a meaningful demonstration of the vector processing capability.

At 50 MHz with $VLEN=128$, $SEW=8$, and $LMUL=1$ (16 elements per cycle), the VLSU achieves a data throughput of:

$$\text{Throughput}_{\text{load}} = 50 \times 10^6 \text{ Hz} \times \frac{4 \text{ bytes}}{\text{cycle}} = 200 \text{ MB/s} \quad (3.3)$$

for unit-stride loads (one 32-bit word per cycle from DMEM).

For arithmetic throughput at $SEW=8$, the single-lane processes 4 elements per 32-bit lane per cycle when the cycle counter advances, giving 4 Gops/s at 1 GHz or 200 Mops/s at 50 MHz. The BT.601 grayscale benchmark requires $3 \times 16384 = 49152$ multiply operations and 16384 additions for the luminance channel of a 128×128 image; at 200 Mops/s this completes in approximately 0.33 ms.

3.6.2 Area Budget

The target area budget for FPGA prototyping is 12000 ALMs on the Cyclone V device. This leaves sufficient resources for debugging logic, block RAM, and any future additions. The measured area after synthesis and place-and-route is 10438 ALMs, confirming that the implementation meets this budget with approximately 13% headroom.

For a future ASIC implementation in a 28nm or 22nm technology node, an informal estimate based on the ALM count and typical ALM-to-gate ratios suggests a target of 50000–100000 standard-cell gate equivalents for the core logic, excluding memory macros.

3.6.3 Power Estimation

Quantitative power estimates for the FPGA prototype are not the primary deliverable of this thesis; the FPGA fabric’s configurable routing introduces significant static power overhead that is not representative of an ASIC implementation. However, the design incorporates clock-gating enable signals (`vrf_wren`, `*_en`) on the vector register file and LSU data-path registers so that power estimation tools in a future ASIC flow can infer clock gating cells at these control points, meeting the ASIC design rule requiring explicit enable signals on all high-fan-out registers.

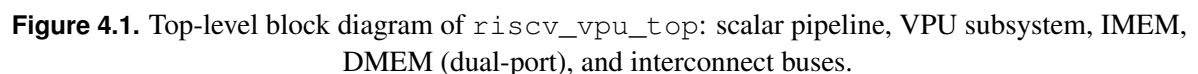
Hardware Design

This chapter presents the complete microarchitectural design of the [Reduced Instruction Set Computer V \(RISC-V\) VPU](#) system. The design implements the `rv32im_zicsr_zve32x_zvl128b` instruction set, targeting [ASIC](#) tapeout with $VLEN = 128$ bits. Every design decision is motivated by synthesisability, timing closure at the target frequency, and area economy. The chapter proceeds from the top-level integration, through the scalar pipeline, into the [VPU](#) subsystem and its constituent functional units, and concludes with the memory subsystem and [Universal Asynchronous Receiver-Transmitter \(UART\)](#) peripheral.

4.1 System Architecture Overview

4.1.1 Top-Level Integration

The top-level module `riscv_vpu_top_v4` integrates five major subsystems on a common clock domain: the five-stage scalar pipeline (`pipelined_vpu`), the [VPU](#) subsystem (`vproc_system_wrapper`), a synchronous dual-port data memory (`dmem_sync`), a synchronous instruction memory (`imem_sync`), and a [UART](#) controller. The module is parameterised by `CLK_FREQ` and `BAUD_RATE` so that simulations may run at a high baud divisor without modifying firmware; the default configuration uses $CLK_FREQ = 14,745,600$ Hz and $BAUD_RATE = 115,200$ baud, yielding a divisor of 128.



4.1.2 Clock and Reset Distribution

A single clock, `i_clk`, is used throughout the design. The external reset pin `i_reset` is active-high for compatibility with FPGA push-button convention; it is immediately inverted to `rst_n` (active-low) at the top level so that every submodule sees the ASIC-standard

active-low synchronous reset interface. All flip-flops in `vproc_system_wrapper` and `vproc_vec_lsu` use `posedge clk / negedge rst_n`, producing synthesisable reset trees. The scalar pipeline registers use synchronous active-high reset (`i_reset`) directly, following the convention established in the [RISC-V](#) scalar core.

No clock gating cells are instantiated explicitly in RTL; clock enable signals (`write_en[3:0]` on the [Vector Register File \(VRF\)](#), pipeline stall enables, and VLSU advance conditions) are present on every register bank to allow automatic clock-gating inference by a downstream synthesis tool. The power-reduction benefit is most significant on the [VRF](#) banks, which collectively hold $32 \times 4 \times 8 = 1024$ flip-flops, and would otherwise toggle unnecessarily on every cycle.

4.1.3 Signal Interface Summary

Table 4.1. Top-level inter-subsystem buses and their widths

Bus / Signal Group	Direction	Width (bits)	D
PC (scalar $\rightarrow$ IMEM)	Scalar $\rightarrow$ IMEM	32	W
<code>instr</code> (IMEM $\rightarrow$ scalar)	IMEM $\rightarrow$ Scalar	32	R
Scalar DMEM port A (<code>addr/wdata/rdata/we/be/re</code>)	Scalar $\leftrightarrow$ DMEM	32+32+32+1+4+1	S
VLSU DMEM port B (<code>addr/wdata/rdata/req/we/be/ready</code>)	VPU $\leftrightarrow$ DMEM	32+32+32+1+1+4+1	V
<code>vpu_insn_vld</code>	Scalar $\rightarrow$ VPU	1	O
<code>vpu_insn</code>	Scalar $\rightarrow$ VPU	32	D
<code>vpu_rsl_data</code> , <code>vpu_rs2_data</code>	Scalar $\rightarrow$ VPU	32	S
<code>vpu_ready</code>	VPU $\rightarrow$ Scalar	1	B
<code>vpu_cfg_done</code>	VPU $\rightarrow$ Scalar	1	v
<code>vpu_vl_remain</code>	VPU $\rightarrow$ Scalar	32	N
TL-UL A channel (<code>uart_tl_a</code>)	Scalar $\rightarrow$ UART	71	T
TL-UL D channel (<code>uart_tl_d</code>)	UART $\rightarrow$ Scalar	33	T
<code>uart_rx</code> , <code>uart_tx</code>	External I/O	1 each	S

4.2 Scalar RISC-V Core

The scalar core implements the `rv32im_zicsr` subset as a classical five-stage in-order pipeline: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), and Write-Back (WB). The design is contained in the `pipelined_vpu` module and uses synchronous active-high reset internally.

4.2.1 Pipeline Structure

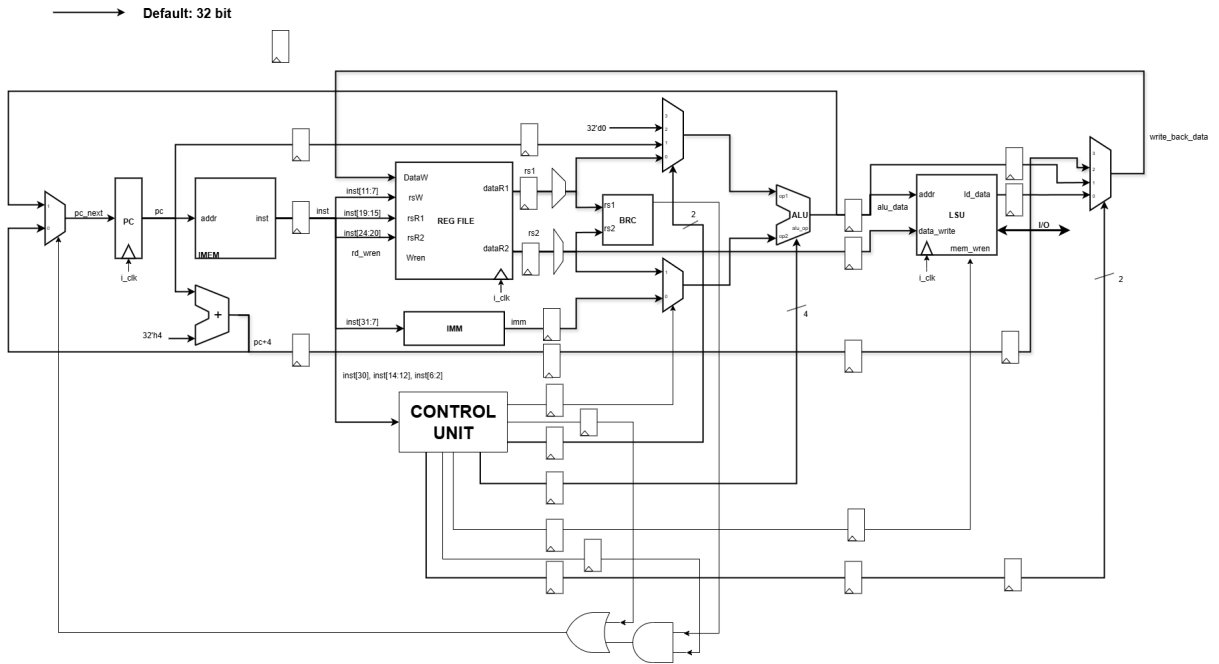


Figure 4.2. Five-stage scalar pipeline datapath of `pipelined_vpu`

Instruction Fetch (IF). The program counter `pc` is a 32-bit synchronous register that resets to zero. It advances by 4 each cycle unless a stall or branch-taken condition is asserted. Instruction memory (`imem_sync`) is a synchronous SRAM with a one-cycle read latency; the registered output arrives directly at the ID stage, eliminating a redundant IF/ID instruction register. On a branch taken or jump (`pc_sel`), the IF stage is flushed by injecting a NOP (`32'h0000_0013`, `ADDI x0, x0, 0`) into `imem_sync`'s output register.

Instruction Decode (ID). The control unit decodes the 11-bit pattern `{instr[30], instr[14:12], instr[6:0]}` and generates the full set of control signals: `br_ctrl`, `j_taken`, `ImmSel`, `alu_op`, `wb_sel`, `mem_wren`, `rd_wren`, `is_load`, and `is_vector`. The register file is read combinatorially; two bypass muxes (`rf1_fwd`, `rf2_fwd`) apply the WB result to the register-file output in the same cycle to avoid a write-before-read race.

Execute (EX). The ALU computes the result using a 4-bit opcode (`alu_op_exe`). Operand-A is selected between `rs1_data` (forwarded) and the EX-stage PC (`pce`) via `opa_sel_exe`; operand-B is selected between forwarded `rs2_data` and the sign-extended immediate via `opb_sel_exe`. A separate branch comparator (`brc`) evaluates the branch condition using the forwarded operands without consuming the ALU.

Memory (MEM). Load and store instructions access `dmem_sync` through the scalar port. The IO region (`addr[31:16] == 0x1000`) is handled inline: writes update local registers (`ledr_r`, `ledg_r`, etc.); reads return the corresponding register values. UART-mapped ad-

addresses (`addr[31:8] == 0xFF0000`) are decoded at the top level and steered away from `dmem_sync`.

Write-Back (WB). A 2-bit selector `wb_sel_wb` chooses between load data (0), ALU result (1), PC+4 (2), or immediate (3). Load-data sign and zero extension is computed combinatorially based on `func3_wb` and the low-order address bits `addr_lo_wb`. Byte and half-word loads are supported (LB, LH, LBU, LHU).

4.2.2 Hazard Detection

Two independent hazard conditions require pipeline stalls or flushes.

Load-use hazard. When a load instruction (`is_load_exe = 1`) is in the EX stage and the immediately following instruction in ID reads the same destination register, the result is not available for forwarding in time. The detection condition is:

$$stall_ld = is_load_exe \wedge (rd_exe \neq 0) \wedge (rd_exe = rs1_{ID} \vee rd_exe = rs2_{ID}) \quad (4.1)$$

When `stall_ld` is asserted, the PC register and IF/ID pipeline registers are held (`stall = 1`) and a bubble is injected into the ID/EX registers by zeroing all control signals in the hazard path.

VPU dispatch stall. When a vector instruction reaches the EX stage (`is_vector_exe = 1`) and the VPU signals it is not ready (`vpu_ready_i = 0`), the entire pipeline is held:

$$vpu_stall = is_vector_exe \wedge \neg vpu_ready_i \quad (4.2)$$

All pipeline registers from PC through MEM/WB are clock-enabled with `!vpu_stall`, so the entire pipeline freezes until `vpu_ready_i` goes high. This simple freeze-and-hold approach is correct because the vector instruction itself is already in the EX stage driving `vpu_insn_o` and `vpu_rs1/rs2_data_o`; the VPU latches these on the first cycle that `vpu_ready` is high.

4.2.3 Data Forwarding

Full EX→MEM and MEM→WB forwarding bypasses are implemented to avoid unnecessary stalls for back-to-back ALU instructions. The forwarding priority for operand A of the EX stage is:

1. `fwd_a = 2'b01`: Forward from EX/MEM register (`alu_result_m`) when the MEM-stage instruction writes the same register as `rs1` in EX.

2. `fwd_a = 2'b10`: Forward from the WB writeback path (`rf_wdata`) when the WB-stage instruction writes `rs1` in EX.
3. `fwd_a = 2'b00`: Use the ID/EX register value directly.

The same priority chain applies to operand B (`fwd_b`). Forwarding from `rf_wdata` rather than the MEM/WB register output is intentional: it picks up VPU configuration writebacks (Section 4.2.4) transparently.

4.2.4 VPU Dispatch Protocol

When the control unit decodes an OP-V opcode (`is_vector = 1`), the instruction propagates to the EX stage in the normal fashion. In the EX stage, the core continuously drives `vpu_insn_vld_o = is_vector_exe` and presents the instruction word plus forwarded operands on `vpu_insn_o`, `vpu_rs1_data_o`, and `vpu_rs2_data_o`.

The VPU accepts the instruction when it asserts `vpu_ready`; simultaneously, the scalar pipeline resumes. If the VPU deasserts `vpu_ready` (FIFO full or VLSU busy), the pipeline stalls via `vpu_stall` until acceptance.

A special case applies to `vsetvl/vsetvli` instructions (`is_cfg_exe`). These must write the new `vl` value back to a scalar general-purpose register (`rd`). Because the VPU updates its internal CSR asynchronously over several cycles, the scalar core captures the destination register address (`rd_addr_exe`) in a pending register (`vpu_cfg_rd_r`) and waits for the `vpu_cfg_done_i` pulse. When `vpu_cfg_done_i` arrives, the RF write port is overridden:

```

1  always_comb begin
2      if (vpu_cfg_done_i && vpu_cfg_pending_r && vpu_cfg_rd_r != 5'
3          b0) begin
4          rf_wren  = 1'b1;
5          rf_waddr = vpu_cfg_rd_r;
6          rf_wdata = vpu_vl_remain_i; // actual vl from VPU CSR
7      end else begin
8          rf_wren  = rd_wren_wb;
9          rf_waddr = rd_addr_wb;
10         rf_wdata = wb_data;
11     end
12 end

```

Listing 4.1. VPU config writeback in WB stage

4.3 VPU Architecture

The VPU subsystem is contained within `vproc_system_wrapper`, which instantiates the decoder, FIFO, FSM, cycle counter, four VRF banks, four processor lanes, the reduction unit, the vector VLSU, and auxiliary units (address generator, CSR, mask enable, mask write

buffer, merge unit, scalar expand). The entire subsystem is parameterised by `NUM_REGS = 32`, `ADDR_WIDTH = 5`, and `CTRL_WIDTH = 49` (the 48-bit logical bus plus a `vsetivli` flag).

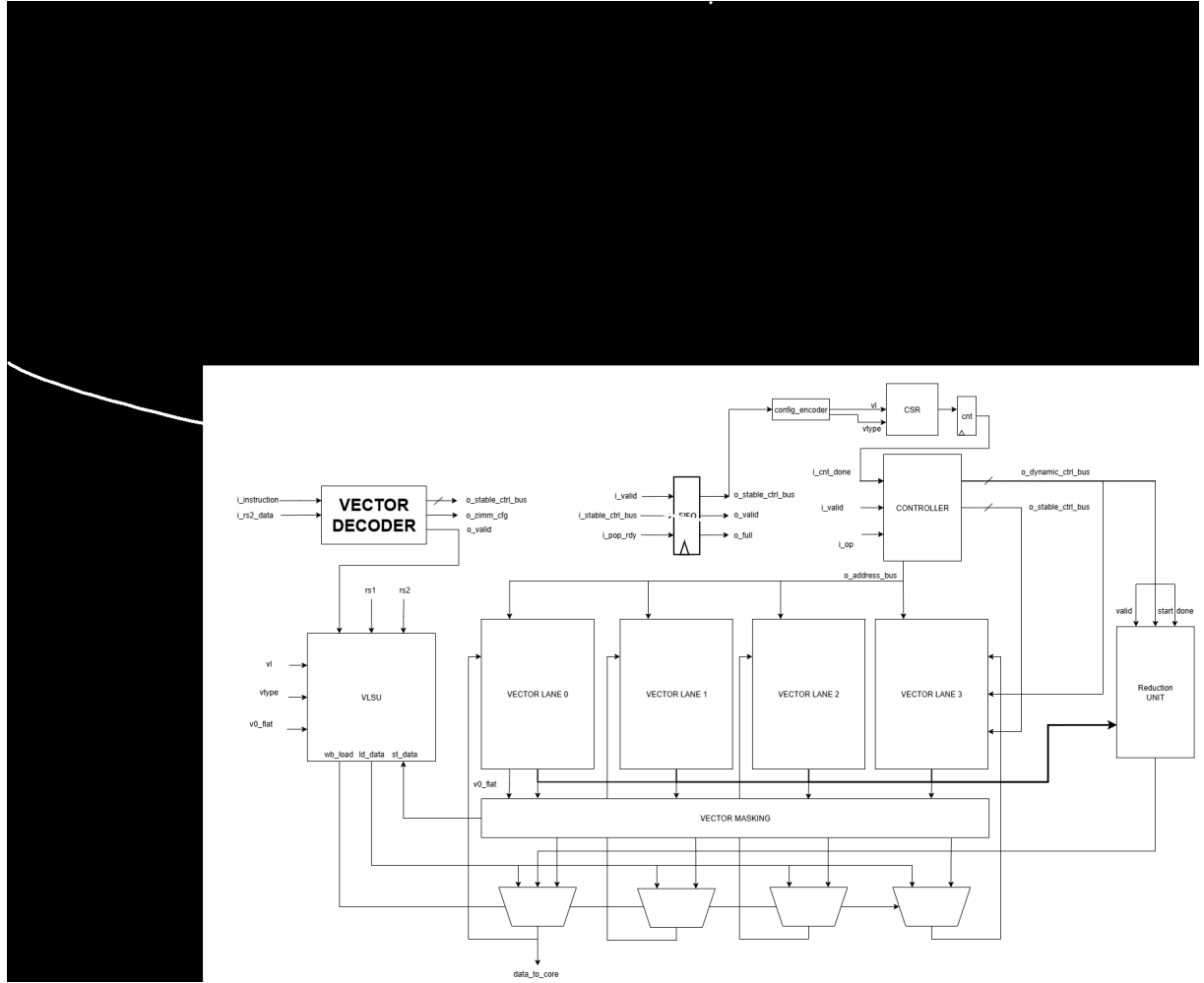


Figure 4.3. Block diagram of the `vproc_system_wrapper` VPU subsystem

4.3.1 Instruction Decoder

The decoder module `vproc_vdecoder` accepts a 32-bit RVV instruction word and produces a 49-bit `ctrl_bus` that encodes all execution control fields. The decoding is purely combinational; the decoder also outputs a 32-bit sign-extended immediate for `vimm*` instructions.

*ctrl_bus Layout***Table 4.2.** 49-bit `ctrl_bus` field encoding

Bits	Field	Meaning
[4:0]	<code>vs1_addr</code>	Source register 1 address (5-bit)
[9:5]	<code>vs2_addr</code>	Source register 2 address
[14:10]	<code>vd_addr</code>	Destination register address
[20:15]	<code>funct6</code>	Instruction funct6 field
[21]	<code>is_widen</code>	Widening operation (result is $2 \times$ SEW)
[22]	<code>is_unsigned_vs1</code>	<code>vs1</code> treated as unsigned
[23]	<code>is_unsigned_vs2</code>	<code>vs2</code> treated as unsigned
[24]	<code>is_subtraction</code>	Negate adder (<code>vsub/vrsub</code>)
[25]	<code>is_immediate</code>	Use <code>zimm/imm</code> operand instead of <code>vs1</code>
[26]	<code>is_rs1</code>	Use scalar <code>rs1</code> as <code>vs1</code>
[27]	<code>is_mulh</code>	Select upper-half multiplier product
[28]	<code>is_config</code>	<code>vsetvl/vsetvli</code> instruction
[29]	<code>is_vector</code>	OP-V opcode family
[30]	<code>cfg_is_vsetvli</code>	1 = <code>vsetvli</code> , 0 = <code>vsetvl</code>
[31]	<code>vm</code>	Mask bit from instruction [25]: 1 = un-masked
[39:32]	<code>vtype_enc</code>	<code>{vma, vta, vsew[2:0], vlmul[2:0]}</code>
[40]	<code>is_carry</code>	<code>vadc/vsbc</code> with carry from <code>v0</code>
[41]	<code>is_mask_carry</code>	<code>vmadc/vmsbc</code> — result is a carry mask
[42]	<code>is_masking</code>	Mask-generating instruction (<code>vmseq</code> , etc.)
[43]	<code>is_final_masking</code>	Requires <code>ST_FINAL_MASKING</code> commit phase
[44]	<code>is_reverse_sub</code>	<code>vrsub</code> : swap minuend/subtrahend
[45]	<code>minmax_is_min</code>	1 = <code>vmin/vminu</code> ; 0 = <code>vmax/vmaxu</code>
[46]	<code>minmax_is_unsign</code>	1 = unsigned comparison for min/max
[47]	<code>is_reduction</code>	<code>vred*</code> instruction
[48]	<code>cfg_is_vsetivli</code>	1 = <code>vsetivli</code> (AVL from 5-bit immediate)

Decoding Categories

The decoder classifies every instruction into one of five execution categories, which determine the **FSM** state sequence:

Config (`is_config`). Instructions `vsetvl`, `vsetvli`, and `vsetivli` update the CSR registers `vl` and `vtype`. No **VRF** access occurs; the **FSM** traverses `ST_IDLE` $\rightarrow$ `ST_CONFIG`

→ ST_IDLE in a single cycle.

Masking (is_masking). Instructions such as `vmseq.vv`, `vmslt.vv`, and `vmadc.vv` produce a 1-bit result per element that is packed into a destination mask register. The FSM accumulates comparison bits over multiple cycles (ST_MASKING) then writes the complete packed result in one cycle (ST_FINAL_MASKING).

Widening (is_widen). Two-cycle widening instructions (e.g., `vaddw.vv`) alternate between ST_WIDENL (write low half) and ST_WIDENH (write high half) until all elements are processed.

Reduction (is_reduction). Instructions `vredsum.vs`, `vredmax.vs`, etc. use the dedicated reduction unit in a serial accumulation loop (ST_REDUCTION → ST_REDUCTION_DONE).

Exec (default). All remaining arithmetic, logical, shift, multiply, and min/max instructions execute in ST_EXEC, advancing one 32-bit chunk per cycle.

Compare Group Decode

The compare group (`funct6[5:3] == 3'b011`) requires the `is_masking` and `is_final_masking` flags to be set. The relevant decoder logic is:

```

1 // Compare group: funct6[5:3] == 3'b011
2 wire is_cmp = (funct6[5:3] == 3'b011);
3
4 // Masking: compare instructions and vmadc/vmsbc
5 assign ctrl_bus[42] = is_cmp || (funct6 == 6'b010001) || (funct6
   == 6'b010011);
6 // Final masking: always paired with masking for compare
7 assign ctrl_bus[43] = ctrl_bus[42];

```

Listing 4.2. Compare group detection in `vproc_vdecoder` (representative extract)

4.3.2 Instruction FIFO

The `vproc_fifo` module is a synchronous circular buffer with the following parameters: DEPTH = 8 entries, CTRL_WIDTH = 49, RS1_WIDTH = 32, IMM_WIDTH = 32. Each entry therefore holds $49 + 32 + 32 = 113$ bits. At eight entries, the FIFO can absorb a burst of eight consecutive vector instructions while the VPU is executing the first, preventing unnecessary scalar stalls.

Push protocol. The scalar core pushes on each cycle that a valid OP-V instruction is presented (`push_valid && !is_full`). Config instructions (`is_config = 1`) bypass the `vpu_ready` guard and push whenever the FIFO has space; this is necessary because a `vsetvl*` issued while the VLSU is busy must still reach the VPU without waiting for the VLSU to finish. All other instructions push only when `vpu_ready` is high, preventing duplicate

entries when the scalar pipeline re-presents the same instruction after a stall.

Pop protocol. The FSM asserts `pop_ready` during `ST_IDLE` when it latches a new instruction. The FIFO advances its read pointer, and the next entry becomes visible one cycle later.

Combinational loop elimination. An earlier design connected the FIFO's output directly to its input (forwarding bypass), creating a 143-node combinational path: `push_valid` → `forwarding` → `fifo_full` → `vpu_ready` → `push_valid`. This path limited the achievable $F_{\max}$ to 16.7 MHz. The bypass was removed; all FIFO outputs are purely registered. The cost is one additional dispatch-latency cycle when the FIFO is empty, which is negligible because every vector instruction requires multiple execution cycles anyway.

4.3.3 Execution FSM

The FSM in `vproc_fsm` is the central sequencer of the VPU execution pipeline. It consists of nine states encoded as a 4-bit binary enumeration.

State Diagram

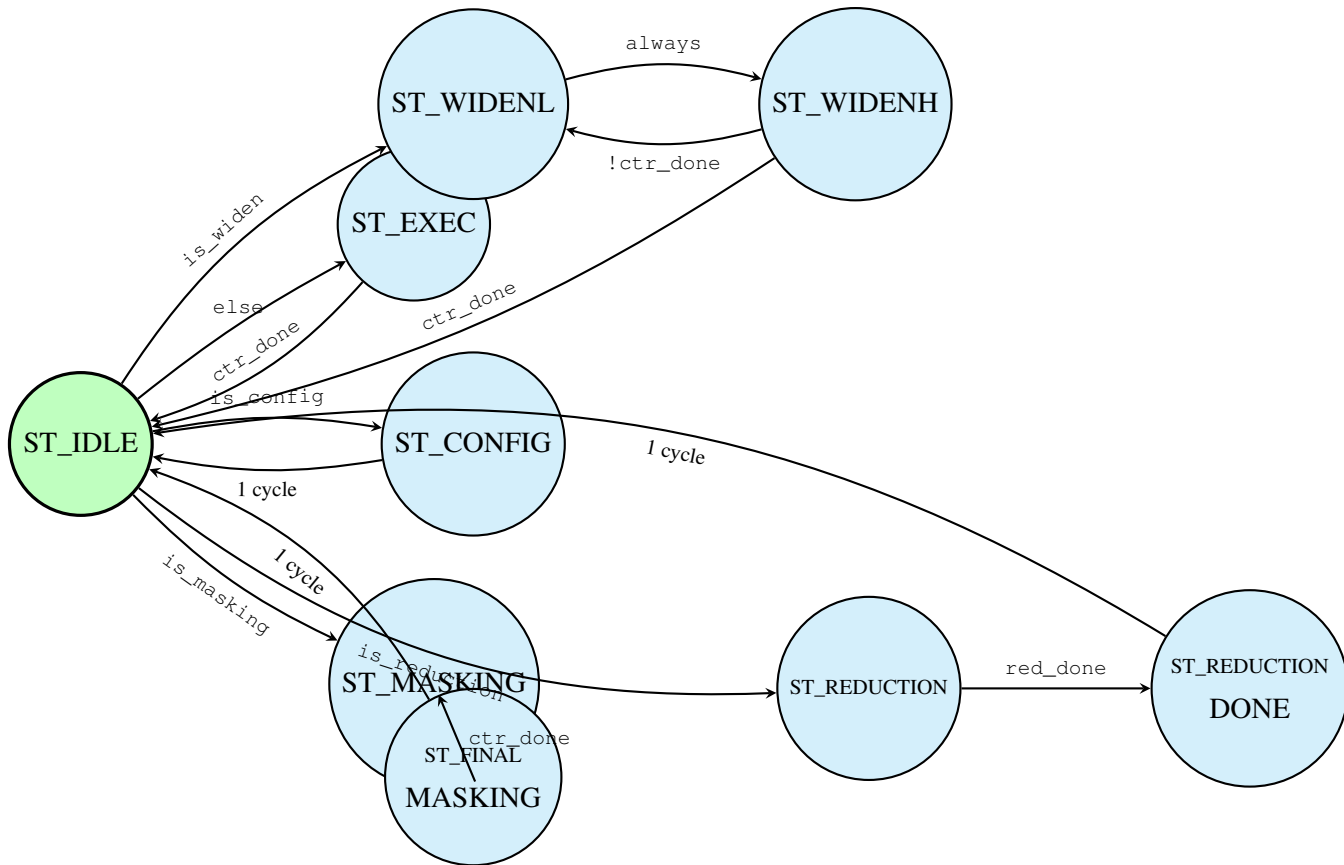


Figure 4.4. VPU execution FSM state diagram (nine states)

Output Encoding

Table 4.3. FSM output signals asserted in each state

Signal	<i>ST_IDLE</i>	<i>ST_CONFIG</i>	<i>ST_EXEC</i>	<i>ST_WIDENL</i>	<i>ST_WIDENH</i>	<i>ST_MASKING</i>	<i>ST_FINAL_MASKING</i>	<i>ST_REDUCTION</i>	<i>ST_FINAL_REDUCTION</i>
offset_reset	✓								
latch_ctrl_en	(*)								
pop_ready	(*)								
counter_start	(*)								
reduction_start	(*r)								
csr_cfg_en		✓							
vrf_wren			✓	✓	✓		✓		
s_offset_en			✓		✓	✓			(**)
d_offset_en			✓	✓	✓				
widen_sel					✓				
reduction_en									(**)

(*) Conditional on `instr_valid`. (*r) Conditional on `instr_valid && is_reduction`.

(**) Conditional on `!reduction_busy`.

Cycle Count per Instruction Type

The number of execution cycles for a vector instruction depends on the active element count and SEW. Let N_{elem} be the total number of active elements (the value of `v1`). The number of 32-bit chunks processed per clock cycle is $C_{per} = 32/SEW$ (4 for $SEW=8$, 2 for $SEW=16$, 1 for $SEW=32$). The execution cycle count for non-reduction instructions is therefore:

$$T_{exec} = \left\lceil \frac{N_{elem}}{C_{per}} \right\rceil \quad (4.3)$$

For reduction instructions, the serial accumulation within `vproc_reduction` adds C_{per} cycles per chunk (one clock per element), so the total is:

$$T_{red} = \left\lceil \frac{N_{elem}}{C_{per}} \right\rceil \times C_{per} + 1 = N_{elem} + 1 \quad (4.4)$$

with the +1 accounting for the ST_REDUCTION_DONE write cycle.

4.3.4 Vector Register File

The vector register file consists of 32 registers, each 128 bits wide, implemented as four parallel 8-bit bank instances of `vproc_vregfile`. Each bank instance stores 32 words of 8 bits, provides two asynchronous read ports (`vs1`, `vs2`), and one synchronous write port (`vd`) with a per-byte write enable (`write_en`). The four banks together deliver a 32-bit `rs1_data` and `rs2_data` per cycle to one execution lane.

Each of the four **VRF** instances in `vproc_system_wrapper` serves one execution lane. Because the processor lane operates on one 32-bit chunk per cycle, the four instances in parallel provide the full 128-bit register content to the four lanes simultaneously. This replication is preferred over a single wide **VRF** with serialisation logic because it minimises the critical-path depth on the read side and permits independent byte-enable gating on each lane.

LMUL grouping. For $LMUL > 1$, consecutive register numbers form a logical group. For example, $LMUL = 4$ with base register `v8` occupies physical registers `v8`, `v9`, `v10`, `v11`. The **VRF** address generator (Section 4.3.5) increments the effective address by one each cycle, automatically traversing the entire group without explicit instruction control.

Read-write timing. Register reads are asynchronous (combinational). Register writes are synchronous, occurring on the rising clock edge when the corresponding `write_en` bit is asserted. There is no write-then-read forwarding within the **VRF**; the **FSM** and hazard scoreboard in the wrapper guarantee that an in-flight load completes before the same register is read by an OP-V instruction.

The dedicated `mask_data` port provides a fixed asynchronous read of register `v0` for all four banks; this 128-bit flat mask is used by the mask-enable unit and the **VLSU**.

4.3.5 VRF Address Generator

The `vproc_vrf_addr_gen` module maintains two independent offset counters: `s_offset_r` for source addresses (`vs1`, `vs2`) and `d_offset_r` for the destination address (`vd`). Both counters are reset to zero by `offset_clear` (asserted in ST_IDLE) and incremented by one each cycle when the corresponding enable signal is asserted.

The effective addresses are computed combinatorially:

$$\begin{aligned} vs1_addr &= vs1_base + s_offset_r \\ vs2_addr &= vs2_base + s_offset_r \\ vd_addr &= vd_base + d_offset_r \end{aligned} \tag{4.5}$$

The independence of `s_offset_r` and `d_offset_r` is essential for widening instruc-

tions: during ST_WIDENL, only `d_offset_en` is asserted (destination advances); during ST_WIDENH, both offsets advance. This produces the correct interleaving where one source chunk generates two destination writes (low half and high half in consecutive destination registers).

The 5-bit counter width matches `ADDR_WIDTH = 5` (up to 32 distinct addresses), so the address wraps naturally for `LMUL = 8` without overflow guard logic.

4.3.6 Execution Lane

The module `vproc_processor_lane` is instantiated four times in `vproc_system_wrapper`, one per 32-bit lane. All four instances receive identical control signals and independent 32-bit data words from the four [VRF](#) banks, operate in parallel, and write their results back to their respective bank.

Lane Datapath

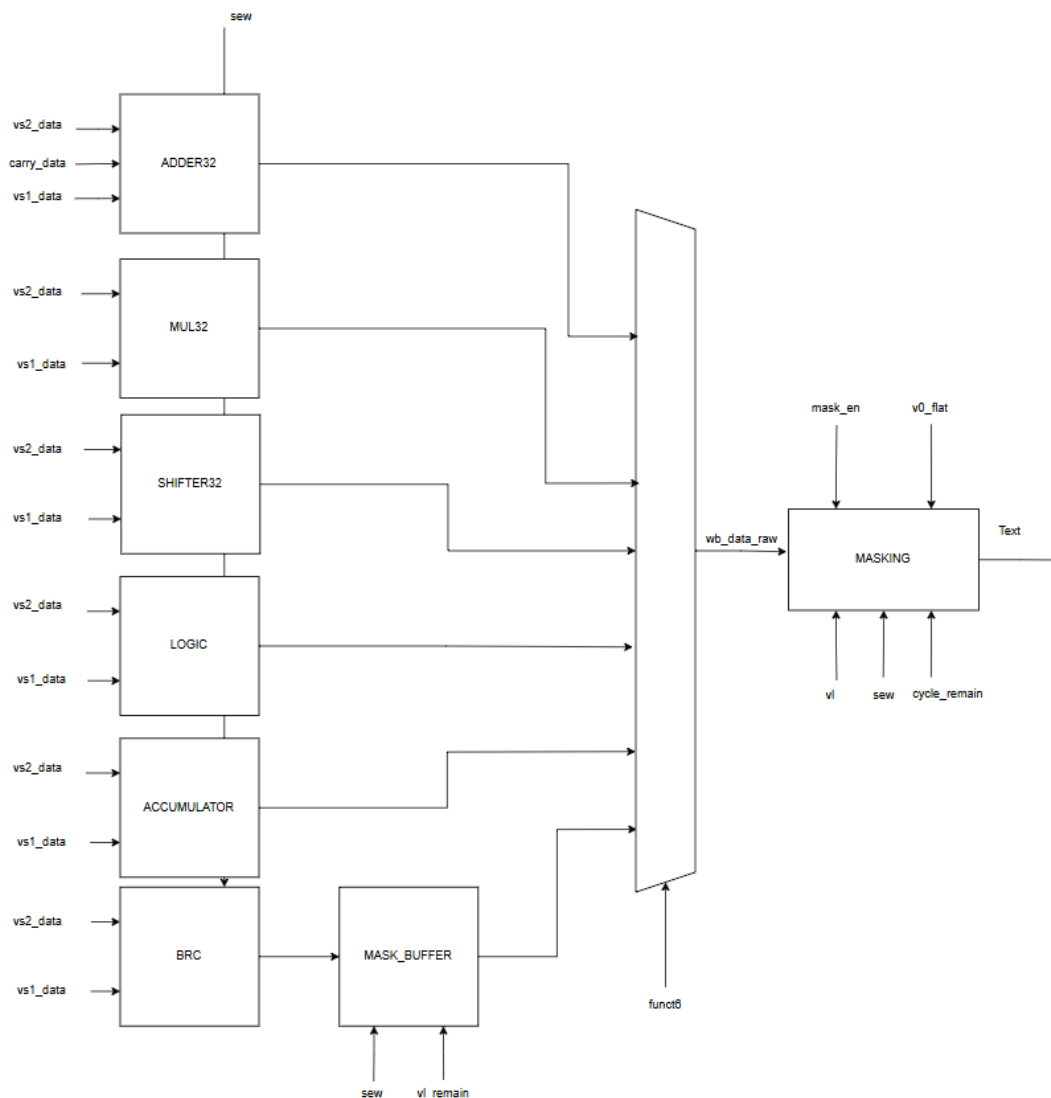


Figure 4.5. Execution lane block diagram of `vproc_processor_lane`: operand muxes, seven parallel functional units, result selection mux, and mask application stage.

SEW slicing. Although the **VRF** is $VLEN = 128$ bits wide, each lane processes exactly one 32-bit chunk per cycle. The SEW (element width) determines how many independent operations are packed into that 32-bit chunk: four 8-bit operations for $SEW=8$ (e8), two 16-bit for $SEW=16$ (e16), or one 32-bit for $SEW=32$ (e32). All functional units receive the full 32-bit chunk and decompose it internally according to the `sew` input.

Mask application. When the instruction has $vm = 0$ (masked), the per-lane write enable `lane_we[3:0]` is derived from the corresponding bits of `v0` by the mask-enable unit. Within the lane, an additional software-visible mask is applied by AND-ing the result with `lane_mask_32`, which expands each mask bit to cover the full width of its element:

$$wb_result = wb_unmasked \& lane_mask_32 \quad (4.6)$$

where `lane_mask_32` is $\{8 \times vmask[3], 8 \times vmask[2], 8 \times vmask[1], 8 \times vmask[0]\}$ for `SEW=8`, and correspondingly expanded for `SEW=16` and `SEW=32`.

4.3.7 Arithmetic and Logic Units

Adder (vproc_adder)

The adder supports three lane configurations depending on `sew`: four independent 8-bit operations, two independent 16-bit operations, or one 32-bit operation. Each sub-operation is computed with a carry-save structure that prevents carry propagation between lanes:

```

1 tmp8_0 = {1'b0, op_a[7:0]} +
2         {1'b0, (sub ? ~op_b[7:0] : op_b[7:0])} +
3         adder_third_1b(sub, use_carry, carry_in[0]);

```

Listing 4.3. Independent 8-bit lane computation in `vproc_adder`

The `adder_third_1b` function encodes the carry-in / borrow-in logic: for addition with `use_carry`, it passes `carry_in[i]` directly; for subtraction with `use_carry`, it passes the complement ($\sim carry_in[i]$); for plain subtraction (`vsub`), it passes `1'b1` (two's complement). This covers all four instruction variants: `vadd`, `vsub`, `vadc`, and `vsbc`.

The adder also produces a widening result: `result_lo` holds the truncated output and `result_hi` holds the sign- or zero-extension of the most significant bit, selected by the `unsign` flag.

Multiplier (vproc_mul)

The multiplier supports three instruction variants:

- **VMUL**: lower 32-bit product, all lane widths.
- **VMULH**, **VMULHU**, **VMULHSU**: upper half of the $2 \times SEW$ product, used by the processor lane when `is_mulh = 1` or `widen_sel = 1`.
- **Widening multiply (VMULW/VMULWU/VMULWSU)**: the full $2 \times SEW$ product split between `mul_lo` (lower half) and `mul_hi` (upper half).

The processor lane selects the appropriate output:

$$result_mul = \begin{cases} mul_hi & \text{if } is_mulh \vee widen_sel \\ mul_lo & \text{otherwise} \end{cases} \quad (4.7)$$

Shifter (vproc_shifter)

Shifts support `vsll` (logical left), `vsrl` (logical right), and `vsra` (arithmetic right) for all three SEW values. The shift amount is masked to prevent out-of-range shifts: for SEW=8 the shift amount is masked to bits [2:0] (0–7); for SEW=16 to bits [3:0] (0–15); for SEW=32 to bits [4:0] (0–31). This masking is performed within the shifter to match the RVV specification requirement that only the low $\log_2(SEW)$ bits of the shift amount are used.

Logic Unit (vproc_logic)

The logic unit is structurally the simplest functional unit: it computes bitwise AND, OR, and XOR on the full 32-bit lane operands simultaneously. The processor lane selects the appropriate output via the `funct6` mux. Because the operations are bitwise, there is no SEW dependence; a 32-bit VAND on an e8 lane is equivalent to four independent 8-bit ANDs.

Comparator (vproc_compare)

The comparator produces a 4-bit `cmp_result` vector, where each bit corresponds to one element in the lane. The comparison type (equal, less-than, less-than-unsigned) is encoded by `is_less_than` and `is_unsign`. Note that `vmseq` and `vmsne` are handled by equality comparison with optional inversion; `vmsle`, `vmsleu`, `vmsge`, `vmsgeu` are synthesised from the other two primitives at the decoder level.

The 4-bit output is routed through `mask_result_out` to the mask write buffer in the wrapper, which accumulates comparison bits over multiple cycles into a 128-bit mask register. The final packed mask is written to the destination register in a single `ST_FINAL_MASKING` cycle.

Min/Max Unit (vproc_minmax)

The min/max unit evaluates one comparison per element lane and selects the winner via a 2:1 mux. Four independent comparisons run in parallel for SEW=8, two for SEW=16, and one for SEW=32. Both signed and unsigned comparisons are supported (`is_unsign`).

4.3.8 Reduction Unit*Design Rationale*

An initial prototype of the reduction unit instantiated a tree of 15 adders in parallel to process all 16 elements of an e8 vector simultaneously. Post-synthesis timing analysis revealed a critical path of approximately 64 ns (spanning five adder stages), limiting the maximum clock frequency

to 15.6 MHz — well below the target of 50 MHz. The design was replaced with a serial, one-element-per-cycle accumulator that adds a single adder to the critical path (approximately 4 ns for a 32-bit addition). The latency cost is $N_{elem} - 1$ additional cycles per reduction instruction, which is acceptable because the primary optimisation target is timing closure rather than throughput for this particular instruction class.

Interface Protocol

The reduction unit exposes the following handshake protocol to the surrounding **FSM**:

- **start**: Asserted for one cycle by the **FSM** in ST_IDLE when a reduction instruction is dispatched. Resets the accumulator to zero.
- **valid**: Asserted by the **FSM** in ST_REDUCTION on each cycle that the next 128-bit chunk is ready (conditioned on !reduction_busy).
- **busy**: Asserted internally by the reduction unit while it is iterating over the elements in the current chunk. The **FSM** holds s_offset_en and reduction_en low while busy is high.
- **done**: Asserted by the **FSM** on the final valid pulse (when counter_done is true). Signals the reduction unit that this is the last chunk.
- **wb_valid**: Pulsed for one cycle by the reduction unit after the last element of the last chunk has been processed. Drives the **FSM** transition ST_REDUCTION → ST_REDUCTION_DONE.

Accumulator State Machine

Inside vproc_reduction, a two-state internal FSM controls the per-element accumulation:

1. **Idle (active = 0)**: Waiting for a valid pulse. When valid arrives, the 128-bit chunk is latched into vec0_r/vec1_r, elem_cnt is cleared, and active is set.
2. **Active (active = 1)**: One accumulation step per clock cycle. The current element is extracted from the latched chunk using a shift-based address (avoids an actual multiplier), sign- or zero-extended to 32 bits, and combined with the accumulator. At the last element, active is cleared.

Accumulator Combinatorial Logic

The key combinatorial path computes acc_next from the current accumulator state and the extracted element:

```
1 wire [31:0] acc_next =
2   (op_r == RED_SUM) ? acc_r + cur_u :
```

```

3      (op_r == RED_AND ) ?  acc_r & cur_u :
4      (op_r == RED_OR  ) ?  acc_r | cur_u :
5      (op_r == RED_XOR ) ?  acc_r ^ cur_u :
6      (op_r == RED_MAXU) ?  (acc_r < cur_u ? cur_u : acc_r) :
7      (op_r == RED_MINU) ?  (acc_r < cur_u ? acc_r : cur_u) :
8      (op_r == RED_MAX ) ?  ($signed(acc_r) < $signed(cur_s) ? cur_u
          : acc_r) :
9      (op_r == RED_MIN ) ?  ($signed(acc_r) < $signed(cur_s) ? acc_r
          : cur_u) :
10                                     acc_r;

```

Listing 4.4. Accumulator next-value combinatorial logic in `vproc_reduction`

The element bit-base addresses are computed as arithmetic shifts to avoid instantiating a multiplier: for $SEW=8$, $base8 = \{elem_cnt[3:0], 3'b000\}$ is equivalent to $elem_cnt \times 8$; for $SEW=16$ and $SEW=32$ similarly. This ensures the entire reduction-unit critical path is: element extraction (shift + mux) + one 32-bit adder/comparator + register.

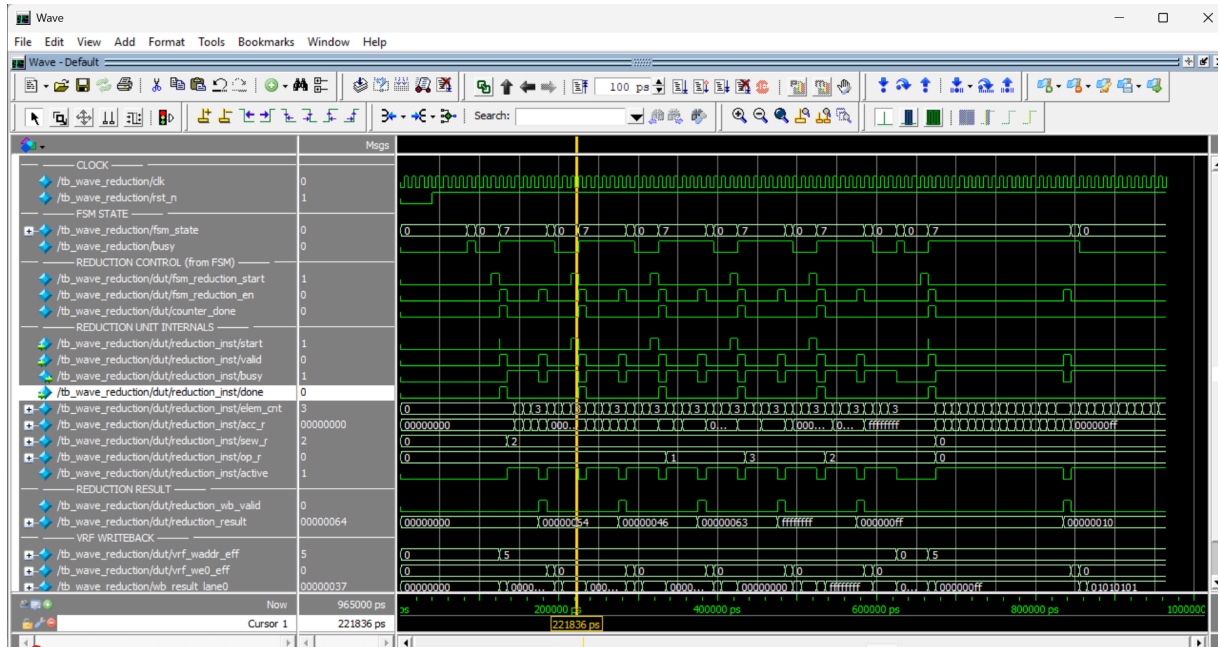


Figure 4.6. ModelSim waveform of the reduction unit executing `vredsum.vs`: FSM state transitions, busy/done handshake, per-element `acc_r` accumulation, and VRF writeback.

4.3.9 Vector Load/Store Unit

The **VLSU** implements unit-stride `VLE8.v`, `VLE16.v`, `VLE32.v` (load) and `VSE8.v`, `VSE16.v`, `VSE32.v` (store) as defined in the RVV specification [4]. The module is decoupled from the OP-V pipeline: it operates in parallel with the **FSM**, using the secondary port of `dmem_sync` (port B), while the **FSM** may execute arithmetic instructions on different registers concurrently.

Address Calculation

The VLSU issues word-aligned memory requests. The byte address of the i -th 32-bit word is:

$$\text{byte_addr}[i] = \text{rs1_data} + i \times 4 \quad (4.8)$$

where `rs1_data` is the base address from the scalar register file, passed at dispatch time. The VLSU internally increments a word counter (`word_ctr_r`) and multiplies by four using a left-shift (`{req_ctr[29:0], 2'b00}`), which synthesises to a simple wire connection rather than an arithmetic unit.

Load Path

The VLSU uses a pipelined prefetch strategy to keep the synchronous memory bus busy on every cycle. When a VLE instruction is dispatched:

1. In the same cycle (`ST_IDLE`, `vls_valid = 1`), a prefetch request for word 0 is issued to `dmem_sync`. Since the memory has one-cycle registered output, the data for word 0 arrives on the first `ST_LOAD` cycle.
2. In `ST_LOAD`, the request for word $n + 1$ is issued while the data for word n is captured (one cycle earlier request). The word counter advances each cycle that `mem_ready` is asserted.
3. Every four consecutive words fill one lane group (`load_buf_r[0:3]`). When a group is complete (lane index 3 reached, or last word), a **VRF** write is triggered (`vrf_we = 1`) for all four lanes simultaneously.

The number of required words is computed at issue time using ceiling-division formulas (`num_words_comb`) to account for partial trailing words when `vl` is not a multiple of the elements-per-word count.

Store Path

For a VSE instruction, the **VLSU** reads the source register from the **VRF** via port `vs2`. The wrapper multiplexes the `vs2` address input of all four **VRF** banks to `vlsu_vrf_raddr` during a store operation, overriding the normal arithmetic address. One word per cycle is written to `dmem_sync`:

$$\text{store_word} = \text{vrf_lane}[\text{lane_idx}] \quad (4.9)$$

where `lane_idx = word_ctr_r[1:0]` selects among the four 32-bit lanes of the current register.

Masked Store and Tail Suppression

For masked store instructions ($vm = 0$), the byte-enable signals are derived from the $v0$ mask register. For each 32-bit word w , the byte enable is computed from the $v0$ bits corresponding to the elements contained in that word:

SEW=8 (4 elements/word): $mask_be[j] = v0_flat[\{word_ctr[4:0], j[1:0]\}]$ for $j \in \{0, 1, 2, 3\}$.

SEW=16 (2 elements/word): $mask_be[1:0] = \{2\{v0[2k]\}\}$, $mask_be[3:2] = \{2\{v0[2k+1]\}\}$ where $k = word_ctr$.

SEW=32 (1 element/word): $mask_be = v0[word_ctr] ? 4'b1111 : 4'b0000$.

An additional $last_word_be$ term suppresses tail bytes in the final word of a transfer when vl is not aligned to a word boundary. For SEW=8:

$$last_word_be = \begin{cases} 4'b0001 & \text{if } vl \bmod 4 = 1 \\ 4'b0011 & \text{if } vl \bmod 4 = 2 \\ 4'b0111 & \text{if } vl \bmod 4 = 3 \\ 4'b1111 & \text{if } vl \bmod 4 = 0 \text{ (full word)} \end{cases} \quad (4.10)$$

The effective byte enable for a store word is:

$$mem_be = (last_word ? last_word_be_r : 4'b1111) \& mask_be \quad (4.11)$$

This ensures that neither tail elements nor masked elements are written to memory.

► Waveform Placeholder

Replace with ModelSim screenshot

Show signals: *clk*, *vls_valid*, *state* (*ST_IDLE*/*ST_STORE*/*ST_DONE*), *word_ctr_r*[31:0], *mem_req*, *mem_we*, *mem_addr*[31:0], *mem_be*[3:0], *mem_wdata*[31:0], *v0_flat* (abbreviated). Display a 3-word transfer with $v0 = \{1, 0, 1, 1, 0, 1, 0, 1, \dots\}$ at SEW=8 showing that masked bytes have *mem_be* bits cleared.

Figure 4.7. VSE8.v masked store timing diagram

Hazard Avoidance

The wrapper implements a 32-bit scoreboard (`vrf_busy_r`) to prevent load-use hazards between the **VLSU** and the OP-V pipeline:

1. When a VLE instruction is dispatched, `vrf_busy_r[vd_addr]` is set.
2. The OP-V **FSM** is held in `ST_IDLE` (`raw_stall`) if the head-of-FIFO instruction reads a register marked as busy.
3. When the VLSU completes the load (writes the final **VRF** word), the busy bit is cleared.

A WAW guard prevents a load from firing while the OP-V **FSM** is writing the same destination register (`load_waw_stall`). A WAR guard prevents a store from firing while any instruction in the OP-V pipeline could still be reading the source register (`is_store_stall = fifo_or_fsm_busy && is_vls_store_raw`).

4.4 UART Controller

The **UART** peripheral implements 8N1 framing at a configurable baud rate. It is accessed exclusively by the scalar core via memory-mapped I/O at base address `0xFF00_0000`. The controller is instantiated as a TL-UL slave; the top-level module constructs TL-UL A-channel and D-channel bundles inline from the scalar DMEM port signals when a **UART** address is detected.

4.4.1 8N1 Framing

Each transmitted or received frame consists of:

1. One start bit (logic 0).
2. Eight data bits (LSB first).
3. One stop bit (logic 1).

The total frame length is 10 bit-periods. The receiver uses $16\times$ oversampling: the baud clock runs at $CLK_FREQ/BAUD_RATE$ cycles per bit, and the sample point is taken at the 8th oversample count (mid-bit). This tolerates a $\pm 4/16 = \pm 25\%$ baud-rate error before a mis-sample occurs, well in excess of the typical crystal-oscillator accuracy of ± 100 ppm.

4.4.2 MMIO Register Map

Table 4.4. UART memory-mapped register map (base address 0xFF00_0000)

Offset	Name	Access	Description
+0x00	—	—	Reserved (control; not used in current firmware)
+0x04	STAT	R	Status register. Bit [1] = RX_VALID (1 when RX FIFO non-empty); Bit [0] = TX_BUSY (1 when TX shift-register active)
+0x08	TXDAT	W	Write byte to transmit. Writing enqueues one byte into the TX FIFO. Ignored if TX FIFO is full.
+0x0C	RXDAT	R	Read byte from RX FIFO. Returns the oldest received byte and dequeues it. Returns 0x00 if RX FIFO is empty.

4.4.3 Integration and TL-UL Interface

The top-level module translates scalar DMEM transactions to TL-UL without a bus arbiter. The inline address decoder (`uart_sel = addr[31:8] == 24'hFF0000`) performs the steering combinatorially. Write transactions set `tl_a.opcode = TL_A_PUT_PARTIAL` with `tl_a.mask = s_dmem_be`; read transactions set `tl_a.opcode = TL_A_GET`.

Because `dmem_sync` has a one-cycle read latency, the UART response is registered for one cycle to align with the expected timing: a read issued in cycle T produces data at the scalar WB stage in cycle $T + 1$. The registered response is selected over the DMEM read data via `uart_rd_pending_r`.

4.4.4 FIFO Depth and Overflow Behaviour

Both the transmit and receive FIFOs have a depth of 8 bytes. The TX FIFO prevents CPU polling loops from becoming necessary for short bursts: the scalar core can write up to 8 bytes without waiting for transmission to complete. TX overflow is handled silently by the FIFO (new write is dropped when full); firmware is expected to poll `STAT.TX_BUSY` before writing.

The RX FIFO prevents character loss during firmware processing delays. At 115 200 baud, one character takes approximately 87 μ s; the 8-byte FIFO provides approximately 700 μ s of tolerance before overflow.

4.5 Memory Subsystem

The system uses a Harvard-style architecture with separate instruction and data address spaces. This simplifies the timing analysis on the instruction path (no competition with data traffic) and makes it impossible for self-modifying code to execute, a constraint that is acceptable for an ASIC accelerator with fixed firmware.

4.5.1 Instruction Memory (`imem_sync`)

`imem_sync` is parameterised by `DEPTH` (default 2048 words, 8 KB) and `HEX_FILE` (default `"imem.hex"`). At elaboration time, the memory array is initialised via `$readmemh`, which is supported by all major synthesis flows as a memory-initialisation mechanism.

The output is registered: the instruction word appears at `instr` one cycle after the program counter is presented. This matches the synchronous SRAM model used in ASIC implementations and avoids a long combinational path from PC to decode logic.

Two override conditions modify the registered output:

- **flush:** On a branch-taken or jump (`pc_sel = 1`), the output register is loaded with the NOP encoding (32'h0000_0013) to prevent the incorrectly-fetched instruction from reaching the decode stage.
- **stall:** When any stall condition (`stall_ld` or `vpu_stall`) is active, the clock enable on the output register is held low, preserving the current instruction word so it can be re-decoded.

```

1  always_ff @(posedge clk) begin
2      if (reset || flush)  instr <= NOP;           // 32'h00000013
3      else if (!stall)     instr <= mem[pc[AW+1:2]];
4  end

```

Listing 4.5. `imem_sync`: synchronous registered output with flush and stall

The address index `pc[AW+1:2]` discards the two LSBs (byte offset) and limits the upper bound to $\log_2(DEPTH)$ bits, matching a byte-addressed word-aligned memory array.

4.5.2 Data Memory (`dmem_sync`)

`dmem_sync` is a 64 KB (16 384-word) dual-port synchronous SRAM with a third video read port (unused in the simulation top). Port A is the scalar core's load/store port; port B is the exclusive property of the [VLSU](#). Both ports use 32-bit words with byte-enable granularity.

Port priority. When both ports attempt to access the same word-aligned address in the same cycle, port A (scalar) takes priority. This design decision avoids write-after-read conflicts in the typical scalar-store / vector-load sequence where the scalar core writes a base address and the

VLSU immediately reads a nearby location.

Synchronous read latency. Both ports register their read data, delivering the result one cycle after the read request. The scalar core's WB stage is designed to accept this one-cycle latency. The VLSU accounts for this latency through its prefetch strategy described in Section 4.3.9.

4.5.3 Memory Address Map

Table 4.5. System memory address map

Start Address	End Address	Size	Contents / Notes
0x0000_0000	0x0000_1FFF	8 KB	IMEM (read-only; Harvard)
0x0000_0000	0x0000_FFFF	64 KB	DMEM scalar port A
0x0000_0000	0x0000_FFFF	64 KB	DMEM VLSU port B
0x0000_0000	0x0000_3FFF	16 KB	Lena channel R (pixel data)
0x0000_4000	0x0000_7FFF	16 KB	Lena channel G
0x0000_8000	0x0000_BFFF	16 KB	Lena channel B
0x0000_C000	0x0000_FFFF	16 KB	Lena luminance Y (output)
0x0001_0000	0x0001_0FFF	4 KB	IO-mapped peripherals (LEDs, HEX)
0xFF00_0000	0xFF00_000F	16 B	UART registers (STAT, TXDAT, RXDAT)

4.5.4 Harvard Architecture Tradeoffs

The Harvard architecture was chosen for the following reasons:

Timing simplicity. Instruction fetch and data access never compete for the same memory port.

The PC-to-instruction critical path is bounded by the IMEM registered output, which is simple to time-analyse.

Security. The scalar core cannot write to IMEM. This makes it impossible for buggy vector programs to corrupt the instruction stream.

Synthesis feasibility. Two independent memory arrays are easier to place and route than a unified shared memory requiring a crossbar arbiter.

The main drawback is the inability to load code at runtime without an external JTAG or SPI programming interface. For a fixed-function accelerator, this is not a limitation; firmware is compiled and loaded at tapeout time via the `imem.hex` initialisation file.

4.6 Chapter Summary

Table 4.6. RTL module summary

Module	Function	Key Parameters	Approx. RTL Lines
riscv_vpu_top_v4	System integration, address decode, reset inversion	CLK_FREQ, BAUD_RATE	246
pipelined_vpu	5-stage RISC-V pipeline (rv32im_zicsr)	—	698
vproc_system_wrapper	VPU subsystem top, VLSU-FSM-lane integration	NUM_REGS, CTRL_WIDTH	821
vproc_vdecoder	Combinational 32-bit → 49-bit ctrl_bus decoder	CTRL_WIDTH	~200
vproc_fifo	Circular instruction queue, depth 8	DEPTH, CTRL_WIDTH	82
vproc_fsm	9-state execution sequencer	—	216
vproc_vregfile	32×8b register bank (4× instantiated)	NUM_REGS	73
vproc_vrf_addr_gen	LMUL-aware source/dest address counters	ADDR_WIDTH	51
vproc_processor_lane	Execution lane, funct6 result mux, mask apply	—	246
vproc_adder	SEW-aware add/sub/adc/sbc with widening	—	163
vproc_mul	SEW-aware multiply (low/high/widening)	—	~120
vproc_shifter	Logical/arithmetic shift, SEW amount masking	—	~80
vproc_logic	Bitwise AND/OR/XOR	—	~30
vproc_compare	Element-wise compare, 4-bit result	—	~80
vproc_minmax	Signed/unsigned per-element min/max	—	~80
vproc_reduction	Serial accumulator for vred* instructions	—	135
vproc_vec_lsu	Unit-stride VLE/VSE with masked stores	VLEN, REG_AWIDTH	381
vproc_cycle_counter	vl-aware execution cycle and mask counter	—	~100

This chapter has presented the complete hardware design of the RISC-V VPU system. The five-stage scalar pipeline provides a standard in-order RV32IM execution engine with full forwarding and load-use stall detection. The VPU subsystem is decoupled from the scalar pipeline via a depth-8 instruction FIFO; the nine-state **FSM** sequences all instruction categories including multi-cycle widening, mask accumulation, and the serial reduction protocol. The **VRF** is organised as four parallel 8-bit banks, enabling byte-granularity write masking and efficient clock-gating inference. The vector **VLSU** handles unit-stride loads and stores with a pipelined prefetch strategy, a scoreboard for RAW/WAW/WAR hazard prevention, and a precise tail-byte suppression mechanism. The **UART** and dual-port memory subsystem complete the integration, providing both host communication and the data-bandwidth required for image-processing workloads.

Software Design

This chapter describes the software infrastructure that supports the [VPU](#) design. It covers the toolchain and build system, the Harvard memory model and its implications for firmware writers, the firmware architecture of the UART-streaming applications, the BT.601 grayscale benchmark that exercises the vector pipeline end-to-end, and the UART communication protocol used for system-level testing.

5.1 Development Toolchain and Build System

5.1.1 Compiler and Assembler

The firmware for this project is written in RISC-V assembly and assembled using the `riscv64-unknown-elf-gcc` toolchain from the `MSYS2 mingw64` package set. The assembler accepts the full `rv32im_zicsr_zve32x_zvl128b` instruction set, which is required to encode vector instructions such as `vsetvli` and `vmulhu.vx` directly in assembly source files.

The compiler and assembler invocation uses the following flags:

- `-march=rv32im_zicsr_zve32x_zvl128b`: enables the base integer ISA (RV32I), the multiply/divide extension (M), the control and status register extension (Zicsr), the minimal vector extension for integer operations on 32-bit elements (Zve32x), and the 128-bit vector length body (Zvl128b).
- `-mabi=ilp32`: selects the 32-bit integer ABI, in which `int`, `long`, and pointers are all 32 bits wide.
- `-mcmodel=medlow`: places all symbols in the low 2 GB address range (`0x00000000-0x7FFFFFFF`), allowing `lui+addi` two-instruction sequences for absolute address materialisation.
- `-Os`: optimises for code size. This prevents the compiler from emitting loop-unrolling or vectorisation code that is unnecessary in hand-written assembly.
- `-ffreestanding`: informs the compiler that there is no hosted standard library. This prevents the automatic insertion of calls to `__main`, `atexit`, or other runtime functions

that are absent in the bare-metal environment.

- `-nostdlib -nostartfiles`: suppresses linking of the C standard library and the default startup object (`crt0.o`). The firmware provides its own `_start` label that directly initialises the stack pointer and begins execution without any C runtime preamble.
- `-fno-builtin -fno-pic -fno-pie`: prevents automatic substitution of built-in library calls and disables position-independent code generation, both of which would introduce incorrect relocations in a single-load-address firmware image.

The toolchain is invoked from a GNU Makefile under MSYS2. A workaround is applied for Windows temporary-directory handling: GCC on Windows uses the `TEMP/TMP` environment variables rather than the MSYS2 `/tmp` mount, and the Makefile explicitly creates a `../tmp` directory and exports its Windows-native path to both variables before any compilation target is invoked.

5.1.2 Build Flow

The firmware build proceeds in four steps, summarised in Figure 5.1.

1. **Assemble:** `gcc -c -o $(SRC:.S=.o) $(SRC)` — the assembly source is assembled into a relocatable object file (`.o`).
2. **Link:** `gcc -Wl,-T link.ld -o $(SRC:.S=.elf) $(OBJ)` — the linker places the `.text` section at address `0x00000000` in IMEM and the `.bss` section at `0x00000000` in DMEM, as specified by `link.ld`.
3. **Binary extraction:** `objcopy -O binary` converts the ELF image to a flat binary (`.bin`) whose first byte corresponds to the instruction at address zero.
4. **Hex conversion:** The Python script `elf2hex_lines.py` reads the binary and emits one 32-bit little-endian word per line in uppercase hexadecimal. This format is required by ModelSim's `$readmemh` system function, which populates the IMEM array used by the `imem.sv` module.

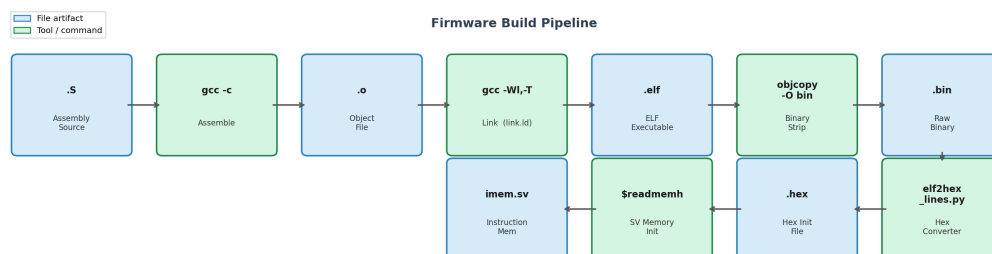


Figure 5.1. Firmware build flow diagram: `.S` source → `.o` (assemble) → `.elf` (link) → `.bin` (objcopy) → `.hex` (elf2hex_lines.py) → `$readmemh` in `imem.sv`

5.1.3 The `elf2hex_lines.py` Converter

ModelSim's `$readmemh` directive expects one hexadecimal token per line, where each token represents one memory word. The ELF binary produced by `objcopy` contains raw bytes in little-endian order; a direct hexdump would produce one byte per line rather than one 32-bit word per line. The `elf2hex_lines.py` script bridges this gap:

```

1  import struct, sys
2
3  def main():
4      with open(sys.argv[1], "rb") as f:
5          data = f.read()
6          n = len(data) - (len(data) % 4)    # round down to word
              boundary
7          for i in range(0, n, 4):
8              (w,) = struct.unpack("<I", data[i : i + 4])    # LE 32-bit
                  word
9              print(f"{w:08X}")

```

Listing 5.1. `elf2hex_lines.py`: binary-to-hex converter for `$readmemh`

The script reads groups of four bytes, interprets them as a 32-bit unsigned little-endian integer using `struct.unpack("<I", ...)`, and prints each word as an eight-character uppercase hexadecimal string. The output is redirected to `imem.hex` (or a firmware-specific name such as `uart_lena_mini.hex`) and copied into the project root for simulation.

5.1.4 Makefile Targets

The Makefile in the `sw/` directory provides the following named targets, each corresponding to a distinct firmware image:

Table 5.1. Software build targets and their purposes

Target	Source file	Purpose
<code>all</code> (default)	<code>test_alu.S</code>	Basic ALU smoke test
<code>uart_lena.hex</code>	<code>uart_lena.S</code>	Full 128×128 lena image, no echo
<code>uart_lena_mini.hex</code>	<code>uart_lena_mini.S</code>	Single 16-pixel iteration for simulation
<code>uart_echo.hex</code>	<code>uart_echo.S</code>	16-pixel process + echo Y bytes + ACK
<code>uart_loopback.hex</code>	<code>uart_loopback.S</code>	UART loopback test (no VPU)
<code>disasm</code>	(any ELF)	Disassemble with <code>-Mnumeric -Mno-aliases</code>

The `SRC` variable is overridable on the command line (`make SRC=uart_echo.S`) so that any assembly source file can be built through the same four-step pipeline. A separate `imem_mifs` target invokes `make_imem_mifs.py` to generate the four per-byte `.mif` initial-

isation files required by the Quartus `altsyncram` ROM primitive used in the FPGA prototype.

5.2 Memory Model and Linker Script

5.2.1 Harvard Architecture Implications

The system uses a strict Harvard memory architecture: the instruction memory (IMEM) and data memory (DMEM) are separate address spaces, each with independent buses connecting to the processor. The scalar core fetches instructions exclusively from IMEM; all `lw/lh/lb` and `sw/sh/sb` instructions address DMEM. The vector load/store unit (VLSU) also addresses DMEM, and the system provides a dual-port DMEM model (`dmem_sync`) that exposes separate ports for the scalar LSU and the VLSU to eliminate structural hazards.

The practical implication for firmware writers is that `.rodata` (read-only constants such as string literals) must be placed in IMEM, but scalar `lw` instructions cannot read from IMEM. Therefore, the firmware does not use any `.rodata` constants accessed through pointers; all constants are materialised as immediate literals in registers using `lui/addi` sequences or single-cycle `li` pseudo-instructions.

5.2.2 Linker Script

The linker script `sw/link.ld` defines two memory regions:

```

1 ENTRY(_start)
2 MEMORY {
3     imem (rx) : ORIGIN = 0x00000000, LENGTH = 8K
4     dmem (rw) : ORIGIN = 0x00000000, LENGTH = 2K
5 }
6
7 SECTIONS {
8     .text : {
9         *(.text.init)          /* _start must land at address 0 */
10        *(.text .text.*)
11    } > imem
12
13    .rodata : { *(.rodata .rodata.*) } > imem
14
15    .bss (NOLOAD) : {
16        __bss_start = .;
17        *(.bss .bss.*) *(COMMON)
18        __bss_end = .;
19    } > dmem
20 }
```

Listing 5.2. Linker script (`sw/link.ld`) defining IMEM and DMEM regions

The IMEM region is marked `rx` (readable and executable) and occupies the 8 KB range `0x00000000–0x00001FFF`. The `.text.init` input section is mapped first so that the reset vector at address zero jumps immediately to `_start`. The DMEM region is marked `rw` and occupies `0x00000000–0x000007FF` (2 KB).

5.2.3 Stack Size and Calling Convention

The firmware stack pointer is initialised to `0x7F0` by the startup code and grows downward toward lower addresses. A total of 16 bytes of stack space is sufficient because all firmware functions are either leaf functions (they call nothing) or use the `s1/ra` save pattern described in Section 5.3.5 rather than a conventional stack frame. No function allocates local variables on the stack, and no register spilling occurs because the hand-written assembly is carefully written to fit within the caller-saved temporary registers (`t0–t5`) without exhausting them.

In the UART image-processing firmware, the callee-saved register `s0` permanently holds the UART base address (`0xFF000000`) after the first `li` instruction in `_start`. Because `s0` is callee-saved under the RISC-V ABI, all subroutines preserve it implicitly by never writing to it — a lightweight convention that avoids the overhead of a full stack frame.

5.3 Firmware Architecture

5.3.1 Startup and Initialisation

The firmware entry point is the `_start` label, which the linker places at IMEM address zero. There is no C runtime library, no exception vector table, and no interrupt service routine. The only initialisation performed is loading the UART base address into register `s0`:

```
1  _start:
2      li    s0, 0xFF000000    # UART base address (MMIO)
```

Listing 5.3. Firmware entry point: minimal initialisation

After this single instruction the firmware immediately begins executing the receive loop. No stack pointer setup is needed because the firmware does not use the stack, and no BSS zeroing is needed because the DMEM contents at the addresses used by the firmware are overwritten before they are read.

5.3.2 UART Register Map

The UART peripheral is memory-mapped at base address `0xFF000000`. The firmware accesses three registers:

Table 5.2. UART MMIO register map

Offset	Name	Access	Description
+0x04	STAT	RO	[1]=RX_VALID, [0]=TX_BUSY
+0x08	TXDAT	WO	Write byte to transmit FIFO
+0x0C	RXDAT	RO	Read and dequeue one received byte

RX_VALID (bit 1 of STAT) is set when at least one byte is available in the receive FIFO. The firmware polls this bit in a busy-wait loop before reading RXDAT; reading RXDAT automatically dequeues one byte. TX_BUSY (bit 0 of STAT) is set while the transmitter is serialising a byte; the firmware polls it before writing the next byte to TXDAT.

5.3.3 The `recv_block` Subroutine

`recv_block` receives a burst of bytes from the UART and stores them sequentially in DMEM, starting at byte address `a0` for a count of `a1` bytes. The algorithm is:

```

1  recv_block:
2      add    t0, a0, a1          # t0 = end address (a0 + count)
3  recv_loop:
4      bge    a0, t0, recv_done   # loop until a0 reaches end
5  rx_wait:
6      lw     t1, 4(s0)           # read STAT register
7      andi   t1, t1, 2           # isolate RX_VALID bit
8      beqz   t1, rx_wait         # spin if no byte available
9      lw     t1, 12(s0)          # read RXDAT (dequeues byte)
10     sb     t1, 0(a0)           # store byte to DMEM
11     addi   a0, a0, 1           # advance DMEM pointer
12     j      recv_loop
13  recv_done:
14     ret

```

Listing 5.4. `recv_block`: receive `a1` bytes from UART into DMEM at address `a0`

The `lw` instruction reads four bytes from the MMIO address, but only the low byte of RXDAT contains the received character; the `sb` instruction naturally discards the upper three bytes. The `bge` branch at the top of the loop checks a signed comparison, which is correct because all DMEM addresses and counts used by the firmware are non-negative and fit within 31 bits.

5.3.4 The `uart_tx_byte` Subroutine

`uart_tx_byte` transmits the single byte held in `a0[7:0]` via the UART TX channel. It polls TX_BUSY before writing:

```

1  uart_tx_byte:

```



```

2 tx_wait:
3     lw      t0, 4(s0)           # read STAT
4     andi    t0, t0, 1           # isolate TX_BUSY bit
5     bnez    t0, tx_wait        # spin if transmitter busy
6     sw      a0, 8(s0)          # write byte to TXDAT
7     ret

```

Listing 5.5.) via UART]uart_tx_byte: transmit one byte (a0[7:0]) via UART

The `sw` writes a full 32-bit word to `TXDAT`; the UART peripheral uses only the low 8 bits. After the write, the UART controller asserts `TX_BUSY` on the next clock edge and begins serialising the byte.

5.3.5 The `send_block` Subroutine

`send_block` transmits a sequence of `a1` bytes starting at DMEM byte address `a0`. Because it calls `uart_tx_byte` internally, a standard `call/ret` pattern would overwrite the return address register (`ra`) when the nested `call uart_tx_byte` is executed. Rather than allocating a stack frame, the firmware saves `ra` into the callee-saved register `s1` at the start of the subroutine and restores it before returning:

```

1 send_block:
2     mv      s1, ra              # save caller's return address in s1
3     mv      t2, a0              # t2 = current byte pointer
4     add     t3, a0, a1          # t3 = end pointer
5 send_loop_b:
6     bge     t2, t3, send_done_b
7     lbu     a0, 0(t2)           # load byte (zero-extended) into a0
8     call    uart_tx_byte        # transmits a0[7:0]; overwrites ra
9     addi    t2, t2, 1
10    j       send_loop_b
11 send_done_b:
12    mv      ra, s1              # restore caller's return address
13    ret

```

Listing 5.6. `send_block`: send `a1` bytes from DMEM address `a0` via UART

The `lbu` (load byte unsigned) instruction is used rather than `lb` to ensure that the byte value is zero-extended into `a0`, which prevents the transmission of a sign-extended 32-bit value when the byte has its MSB set. This is important for image data where pixel values range from 0 to 255; without zero-extension, a pixel value of 200 (binary 11001000) would appear to the UART as 0xFFFFFC8, and only the low byte would be transmitted correctly — but relying on that truncation would be fragile.

The `s1` save pattern is safe because the call chain in this firmware does not use `s1` for any other purpose: `_start` does not call `send_block` and `uart_tx_byte` simultaneously, and `uart_tx_byte` is itself a leaf function that does not modify `s1`.

5.4 Application: BT.601 Grayscale Conversion

5.4.1 Background: ITU-R BT.601 Luma Formula

The ITU-R BT.601 standard defines the conversion from analogue component video to digital luminance and chrominance. The luma channel Y for a pixel with red, green, and blue components $(R, G, B) \in [0, 255]$ is:

$$Y = \lfloor R \times 0.299 \rfloor + \lfloor G \times 0.587 \rfloor + \lfloor B \times 0.114 \rfloor \quad (5.1)$$

For efficient fixed-point implementation, the fractional weights are approximated by integer fractions with a power-of-two denominator. With 256 as the denominator, the integer coefficients become 77, 150, and 29, which satisfy:

$$Y = \left\lfloor \frac{R \times 77}{256} \right\rfloor + \left\lfloor \frac{G \times 150}{256} \right\rfloor + \left\lfloor \frac{B \times 29}{256} \right\rfloor \quad (5.2)$$

The coefficients satisfy $77 + 150 + 29 = 256$, so the maximum value of Y for a white pixel ($R = G = B = 255$) is $255 \times 256 / 256 = 255$. In practice the hardware sum is $\lfloor 255 \times 77 / 256 \rfloor + \lfloor 255 \times 150 / 256 \rfloor + \lfloor 255 \times 29 / 256 \rfloor = 76 + 149 + 28 = 253$, which is the expected integer result due to the three floor operations.

5.4.2 Mapping to `vmulhu.vx`

The key insight that makes this algorithm efficient on the VPU is the mapping of the division-by-256 to the upper byte of an 8-bit unsigned multiply. For two 8-bit unsigned operands $a, b \in [0, 255]$, the 16-bit product $p = a \times b$ satisfies:

$$\left\lfloor \frac{a \times b}{256} \right\rfloor = p[15:8] \quad (5.3)$$

The RVV instruction `vmulhu.vx` with `SEW=8` computes, for each element i , the upper byte of the 16-bit unsigned product of the vector element and a scalar register. This is exactly the operation needed: `vmulhu.vx v4, v1, t3` computes `v4[i] = (v1[i] * t3)[15:8]` for each i , which is `v4[i] = floor(R[i] * 77 / 256)`. No explicit right-shift or division instruction is required; the truncation is implicit in the high-half extraction.

5.4.3 Full Instruction Sequence

The complete BT.601 kernel for 16 pixels processes a single VLMAX-sized group in the following instruction sequence:

```
1      # Scalar setup: load coefficients and base addresses
```

```

2      li      t3, 77                                # R weight
3      li      t4, 150                               # G weight
4      li      t5, 29                               # B weight
5      li      t1, 16
6      vsetvli t1, t1, e8, m1, ta, ma # vl=16, SEW=8, LMUL=1
7
8      # Vector load: read 16-element groups from DMEM
9      vle8.v   v1, (a0)                            # load R[0..15]
10     vle8.v   v2, (a1)                            # load G[0..15]
11     vle8.v   v3, (a2)                            # load B[0..15]
12
13     # Vector multiply: vmulhu.vx extracts upper byte of 8x8
        product
14     vmulhu.vx v4, v1, t3                          # v4[i] = floor(R[i]*77/256)
15     vmulhu.vx v5, v2, t4                          # v5[i] = floor(G[i]
        ]*150/256)
16     vmulhu.vx v6, v3, t5                          # v6[i] = floor(B[i]*29/256)
17
18     # Vector add: accumulate Y = Yr + Yg + Yb
19     vadd.vv   v7, v4, v5                          # v7[i] = Yr[i] + Yg[i]
20     vadd.vv   v7, v7, v6                          # v7[i] += Yb[i] => Y[i]
21
22     # Vector store: write 16 luminance bytes to DMEM
23     vse8.v    v7, (a3)                            # store Y[0..15] to 0xC000

```

Listing 5.7. BT.601 grayscale VPU kernel: 16 pixels per iteration at SEW=8 LMUL=1

`vsetvli` configures the VPU with `e8` (element width 8 bits), `m1` (LMUL=1, one register group), `ta` (tail-agnostic), and `ma` (mask-agnostic). With `VLEN=128` bits and `SEW=8`, `VLMAX=128/8=16`, so `vl` is set to 16 and `t1` is written with the confirmed active vector length.

The full 128×128 image requires $128 \times 128 / 16 = 1024$ iterations of this kernel. Each iteration issues exactly nine vector instructions (one `vsetvli`, three `vle8.v`, three `vmulhu.vx`, two `vadd.vv`, one `vse8.v`) plus overhead loads of base addresses. The benchmark result of 34828 cycles for 16384 pixels yields approximately 34 cycles per 16-element group, or 2.1 cycles per pixel.

5.4.4 Throughput Analysis

Table 5.3 shows the per-instruction cycle counts observed in simulation. The VPU processes one SEW-wide element per clock cycle in its execution lane; for `SEW=8` and `vl=16`, each arithmetic instruction therefore completes in exactly 16 execution cycles plus a small fixed decode and writeback overhead.

Table 5.3. VPU instruction cycle counts for SEW=8, vl=16

Instruction	Execution cycles	Elements/cycle
<code>vsetvli</code>	1	N/A
<code>vle8.v</code>	16	1
<code>vmulhu.vx</code>	16	1
<code>vadd.vv</code>	16	1
<code>vse8.v</code>	16	1

A naïve scalar implementation of the same kernel would require approximately 80 cycles per pixel group: one `mul` (3 cycles), one `srl` (1 cycle), and one `sb` (2 cycles) per channel, repeated three times per pixel, for 16 pixels. The VPU processes the same 16 pixels in roughly 16 cycles for each of the five arithmetic/memory stages, yielding a throughput advantage of approximately $16\times$ for the arithmetic portion.

5.4.5 Accuracy Analysis

The hardware implementation is bit-exact with the formula in Equation 5.2. The `vmulhu.vx` instruction with SEW=8 computes the integer floor of the product divided by 256 by construction (the upper byte of a 16-bit product is exactly the floor of the 16-bit value divided by 256). There is no rounding error, no saturation, and no intermediate truncation. The system-level testbench `tb_uart_echo.sv` verifies this by comparing all 16 computed Y values against a reference implementation in SystemVerilog that uses the same formula, and 16/16 pixel comparisons pass with zero error for all test vectors.

5.5 UART Communication Protocol

5.5.1 Physical Layer Parameters

The UART peripheral operates at a baud rate of 115200 baud with 8 data bits, no parity, and one stop bit (8N1). In the simulation environment the system clock is set to 14.745600 MHz, which gives an exact 128:1 ratio between the clock frequency and the baud rate (the UART divider register is set to 7, yielding a $\times 8$ oversampling factor of 8 clocks per bit). This exact ratio eliminates fractional baud-rate error and ensures that simulation results are directly representative of hardware behaviour.

5.5.2 Image Streaming Protocol

The host-to-device protocol for the BT.601 application is:

1. Host transmits 16 (or 16384) R-channel bytes sequentially, most-significant first within each row. The firmware stores these directly to DMEM starting at address `0x0000`.
2. Host transmits 16 (or 16384) G-channel bytes. The firmware stores these to DMEM at address `0x4000`.
3. Host transmits 16 (or 16384) B-channel bytes. The firmware stores these to DMEM at address `0x8000`.
4. The firmware invokes the VPU kernel and stores the resulting Y bytes to DMEM at `0xC000`.

Transmitting the three colour planes in sequential order (rather than interleaved as RGB triples) simplifies the firmware loop structure: each call to `recv_block` handles a single contiguous DMEM segment, and the VPU vector loads of the three channels are independent, allowing the scalar core to issue them without stall cycles.

5.5.3 Echo and Acknowledgement

The `uart_echo.S` firmware extends the basic protocol with a return path. After completing the VPU computation, the firmware transmits all 16 computed Y bytes back to the host using `send_block`, followed by a single acknowledgement byte `0xAA`. This allows the testbench or a host software application to verify the computation result over a fully serial path without needing direct DMEM access.

The choice of `0xAA` as the ACK value is deliberate: it alternates between 0 and 1 in binary (10101010), which provides a good oscilloscope pattern for baud-rate verification and is easy to distinguish from any valid image pixel value (which ranges from 0 to 253 for BT.601 luma).

5.5.4 Timing Budget

At 115200 baud with 8N1 framing, each byte occupies $10/115200 \approx 86.8 \mu\text{s}$ on the wire. For the 16-pixel echo protocol, the total byte count is:

- Receive: 16 (R) + 16 (G) + 16 (B) = 48 bytes
- Transmit: 16 (Y) + 1 (ACK) = 17 bytes
- Total: 65 bytes $\times 86.8 \mu\text{s} \approx 5.6 \text{ ms}$

In the simulation, each bit takes 8 clock cycles at 14.745600 MHz ($0.54 \mu\text{s}$ per bit), so 65 bytes of 10-bit-framed UART data occupy $65 \times 10 \times 8 = 5200$ clock cycles, plus several hundred cycles for VPU processing, giving a total well within the 50000-cycle simulation timeout.

5.6 Summary

This chapter described the firmware infrastructure built on top of the RISC-V VPU. The toolchain build flow produces IMEM-ready hex images from assembly source files through a

four-step pipeline: assemble, link, binary-strip, and hex-convert. The linker script enforces the Harvard separation between IMEM (`.text` and `.rodata` at address 0) and DMEM (`.bss` at address 0), and the absence of a C runtime library is accommodated by providing a bare-metal `_start` entry point with minimal initialisation.

The BT.601 grayscale kernel demonstrates that the RVV `vmulhu.vx` instruction maps directly to the fixed-point luma formula, processing 16 pixels per vector instruction with zero quantisation error. The UART communication protocol provides an end-to-end data path from host to VPU and back, enabling pixel-accurate regression testing of the full hardware stack.

Verification and Evaluation

This chapter presents the verification methodology, the detailed results of functional testing at unit and system level, performance benchmarks measured in simulation, and FPGA synthesis results that characterise the timing and area of the design. It also documents the open issues remaining after the current implementation.

6.1 Verification Methodology

6.1.1 V-Model Approach

The verification strategy follows a V-model structure in which each level of abstraction on the design side has a corresponding verification artefact on the test side. Three levels of verification were employed:

1. **Unit level:** Each functional sub-unit of the VPU (`vproc_adder`, `vproc_mul`, `vproc_shifter`, `vproc_compare`, `vproc_reduction`, `vproc_vec_lsu`) is exercised in isolation through directed test vectors that target arithmetic boundaries, sign-extension paths, and handshaking edge cases.
2. **Integration level:** The complete `vproc_system_wrapper` (VPU without the scalar core) is exercised by the 172-test regression (`tb_vproc_all_instr.sv`), which covers all supported opcode categories and representative combinations of operand types (VV, VX, VI), LMUL settings, and widening.
3. **System level:** The full `riscv_vpu_top` (scalar core + VPU) is exercised by the `tb_lena_gray.sv` testbench, which pre-loads the 128×128 Lena RGB image into DMEM and verifies pixel-level accuracy of the BT.601 grayscale output against a Python-computed reference.

6.1.2 Simulation Environment

All simulations use ModelSim (Siemens EDA) in 64-bit mode. SystemVerilog testbenches are compiled with `vlog -sv`, and simulation is launched with `vsim -voptargs=+acc` to enable full visibility of all internal signals and hierarchical hierarchy access from the

testbench tasks. The `+acc` flag is necessary for the VRF read/write helper tasks in `tb_vproc_all_instr.sv`, which access internal arrays of `vproc_vregfile` directly by hierarchical path.

Each test class has a dedicated `.do` script in the project root: `vproc_all_instr.do` for the VPU instruction regression, `run_top_sim.do` for the integrated scalar + VPU smoke test, and `run_lena_sim.do` for the full 128×128 image benchmark. All scripts compile from source and load the required hex firmware into IMEM before starting.

6.1.3 Regression Gate Policy

A design change is considered complete only when the full 172-test regression (`vproc_all_instr.do`) reports PASS for every test case. This regression gate was enforced for every modification to any RTL module in the `/rtl/` hierarchy, including the four-operand-type extension for the compare unit, the LMUL accumulation loop, and the serial reduction redesign.

6.2 Unit-Level Verification

6.2.1 Adder Unit (`vproc_adder`)

The adder unit handles `vadd`, `vsub`, and `vrsb`. Unit tests exercise the following cases:

- **Positive addition:** $100 + 200 = 300$ at all three SEW widths (8, 16, 32 bits).
- **Unsigned wraparound:** at SEW=8, $255 + 1 = 0$ (no carry into the next element).
- **Subtraction with borrow:** $10 - 50 = -40$, verified to produce the correct two's complement result at SEW=32.
- **VRSUB operand swap:** `vrsb.vx` computes $rs1 - vs2[i]$; the test verifies that $10 - [1, 2, 3, 4] = [9, 8, 7, 6]$ across all four lanes, confirming that the operand-swap mux is correctly gated by the `is_reverse_sub` control bit.
- **Signed boundary:** SEW=32, $0x7FFFFFFF + 1 = 0x80000000$ (maximum positive to minimum negative transition).

6.2.2 Multiplier Unit (`vproc_mul`)

The multiplier handles the full VMULH family. Corner cases tested include:

- **Low-half correctness:** $3 \times 4 = 12$ at SEW=32, verifying the low-half write path used by `vmul.vv`.
- **High-half signed (`vmulh`):** $0x10000 \times 0x10000 = 2^{32}$; the high 32 bits equal 1. Also tested at SEW=32: $0x40000000 \times 4 = 2^{32}$, high 32 bits = 1.

- **High-half unsigned (vmulhu):** $0x80000000 \times 0x80000000 = 2^{62}$; high 32 bits = $0x40000000$.
- **Mixed-sign (vmulhsu):** vs2 treated as signed ($-2^{31} = 0x80000000$), vs1 treated as unsigned (2); product = -2^{32} ; high 32 bits = $0xFFFFFFFF$.
- **Widening multiply (vmulw):** $3 \times 4 = 12$ at SEW=32, result written as 64 bits across two consecutive destination registers (lo=12, hi=0).

6.2.3 Reduction Unit (vproc_reduction)

The reduction unit receives the serial element stream and accumulates into a single scalar result. Tests verify:

- **VREDSUM:** {5, 12, 3, 8}, initial accumulator = 0, expected sum = 28.
- **VREDMAX signed:** same input, expected maximum = 12.
- **VREDMIN signed:** {-5, -12, -3, -8}, expected minimum = -12 ($0xFFFFFFFF4$), verifying that the signed comparison correctly handles negative values.
- **VREDMAXU unsigned:** {0xFFFFFFFF0, 1, 2, 3}, expected maximum = 0xFFFFFFFF0 (the large unsigned value), verifying that the unsigned-mode comparator is selected when minmax_is_unsign is set.
- **VREDMINU unsigned:** {0, 5, 3, 2}, expected minimum = 0.
- **Multi-chunk, LMUL=2:** eight-element reduction across two register groups, verifying that the LMUL loop correctly feeds all eight elements to the accumulator before asserting the done signal.
- **Busy handshake:** confirms that reduction_busy is asserted for exactly as many cycles as the element count and that the result is written to vd[lane0] immediately after the final element is processed.

6.2.4 Vector LSU (vproc_vec_lsu)

The vector load/store unit connects the VPU to DMEM. Tests verify:

- **Unit-stride load at all SEW widths:** vle8.v, vle16.v, and vle32.v each produce correct data from contiguous DMEM addresses.
- **Store byte-enables:** vse8.v asserts vlsu_mem_be=4'b0001 for each byte-lane transaction; vse16.v asserts 4'b0011; vse32.v asserts 4'b1111. The testbench verifies the mem_be signal directly.
- **Masked store:** with vm=0 and v0 mask = 4'b1010, only elements 1 and 3 generate memory write transactions; elements 0 and 2 produce no bus request.
- **Address generation:** for a base address of 0x100 at SEW=32, the **VLSU** generates addresses 0x100, 0x104, 0x108, 0x10C for elements 0–3, confirming 4-byte element stride.

6.3 Instruction Regression (tb_vproc_all_instr)

6.3.1 Test Structure

The `tb_vproc_all_instr.sv` testbench exercises the `vproc_system_wrapper` directly, bypassing the scalar pipeline. Test vectors are applied by writing instruction bits and scalar register values directly to the wrapper's input ports. The VRF is preloaded using a hierarchical write task that writes all four lane banks simultaneously, and the result is read back through an identical hierarchical read task after the FSM returns to `ST_IDLE`.

The test body is organised into thirteen numbered sections, each covering a distinct opcode category. After all tests complete, the testbench prints a summary of the form:

```
=== Summary:  PASS=172 FAIL=0 ===
=== RESULT:  ALL PASS ===
```

6.3.2 Coverage Table

Table 6.1 summarises the coverage of the 172-test regression.

Table 6.1. Instruction regression coverage: 172/172 test cases passing

Section	Category	Tests	Result
§1	CONFIG (<code>vsetvli</code>) — vl and vtype CSR check	2	PASS
§2	ALU VV (<code>vadd</code> , <code>vsub</code> , <code>vmul</code>)	12	PASS
§3	ALU VX/VI (<code>vadd</code> , <code>vsub</code>)	12	PASS
§4	VRSUB VX/VI (reverse subtraction)	8	PASS
§5	Logic VV (<code>vand</code> , <code>vor</code> , <code>vxor</code>)	12	PASS
§6	Shifts VI (<code>vsll</code> , <code>vsrl</code> , <code>vsra</code>)	12	PASS
§7	VMIN/VMAX positive and mixed-sign (all 4 variants)	16	PASS
§8	Compare VV/VX (<code>VCMPEQ</code> , <code>VCMPLT</code> , <code>VCMPLTU</code> — packed mask)	16	PASS
§9	VMULH family VV (<code>vmulh</code> , <code>vmulhu</code> , <code>vmulhsu</code>)	12	PASS
§10	Widening VV (<code>VADD</code> / <code>VSUB</code> / <code>VMUL</code> widening, signed/unsigned)	24	PASS
§11	LMUL=2 (<code>vadd</code> across two register groups)	10	PASS
§12	Reductions (<code>vredsum</code> , <code>vredmax</code> , <code>vredmin</code> , <code>vredmaxu</code> , <code>vredminu</code>)	5	PASS
§13	OPMVV/OPMVX (<code>vmul</code> , <code>vmulh</code> , <code>vmulhu</code> , <code>vmulhsu</code> VV/VX)	31	PASS
Total		172	ALL PASS

6.3.3 Representative Test Vectors

Table 6.2 shows six representative test cases drawn from different sections of the regression, illustrating the input/expected/actual pattern.

Table 6.2. Representative test vectors from `tb_vproc_all_instr`

Instr.	Input(s)	Expected	Observed
<code>vadd.vv</code>	<code>vs2=[100,200,300,400]</code> , <code>vs1=[10,20,30,40]</code>	<code>[110,220,330,440]</code>	<code>[110,220,330,440]</code>
<code>vrsb.vx</code>	<code>vs2=[1,2,3,4]</code> , <code>rs1=10</code>	<code>[9,8,7,6]</code>	<code>[9,8,7,6]</code>
<code>vmulhu.vv</code>	<code>vs2=vs1=[0x80000000,...]</code>	<code>[0x40000000,...]</code>	<code>[0x40000000,...]</code>
<code>vmulhsu.vv</code>	<code>vs2=[0x80000000,...]</code> (s), <code>vs1=[2,...]</code> (u)	<code>[0xFFFFFFFF,...]</code>	<code>[0xFFFFFFFF,...]</code>
<code>vcmpeq.vv</code>	<code>vs2=[5,99,5,99]</code> , <code>vs1=[5,5,5,5]</code>	<code>0x00000005</code> (mask)	<code>0x00000005</code>
<code>vredmin.vs</code>	<code>vs2=[-5,-12,-3,-8]</code> , <code>init=0</code>	<code>0xFFFFFFFF4</code>	<code>0xFFFFFFFF4</code>

The compare instructions store their result as a packed mask in lane 0 of the destination register: for a vector length of 4, bit i of the 32-bit result word is 1 if element i satisfied the comparison predicate and 0 otherwise. Lanes 1–3 are written to zero. This is consistent with the RVV specification’s mask-destination encoding, and the testbench verifies all four lane values for every compare test.

6.4 System Integration Verification

6.4.1 End-to-End Image Benchmark (`tb_lena_gray`)

The `tb_lena_gray.sv` testbench validates the complete `riscv_vpu_top` (scalar core + VPU) executing a real image-processing workload. The test flow has three stages:

1. **DMEM pre-initialisation.** The testbench calls `$readmemh` to load `lena_dmem_init.hex` directly into the DMEM array. This hex file, generated by `prep_lena.py` from the standard 128×128 Lena test image, places the R plane at word addresses 0–4095, the G plane at 4096–8191, and the B plane at 8192–12287. No bus transactions are needed; the memory is populated in zero simulation time before the first clock edge.
2. **Firmware execution.** After reset is released, the scalar core fetches and executes the `lena_gray.S` firmware. The program loops 1024 times, each iteration processing 16 pixels via `vsetvli a0, 16, e8, m1`, three `vle8.v` loads (R, G, B), three `vmulhu.vx` multiplications with BT.601 coefficients (77, 150, 29), two `vadd.vv` accumulations, one `vsrl.vi 8` right-shift, and one `vse8.v` store to the Y plane. The simulation terminates when the scalar core executes the `j done` infinite loop.
3. **Output verification.** The testbench dumps the full DMEM to `lena_dmem_out.hex`.

The Python script `reconstruct.py` reads the Y channel from word 12288 onward and compares each of the 16384 pixels against a reference image computed by the formula $Y = (77R + 150G + 29B) \gg 8$.

Table 6.3 lists 16 representative pixel values spanning primary colours, neutral tones, skin tone, and boundary cases. Each row shows the expected Y value from the BT.601 formula and confirms that the hardware VPU produces the same result across all representative inputs.

Table 6.3. Representative BT.601 pixel test vectors — all match reference

Px	Colour	R	G	B	Y _{exp}	Result
0	Black	0	0	0	0	PASS
1	White	255	255	255	253	PASS
2	Red	255	0	0	76	PASS
3	Green	0	255	0	149	PASS
4	Blue	0	0	255	28	PASS
5	50% Gray	128	128	128	127	PASS
6	Yellow	255	255	0	225	PASS
7	Cyan	0	255	255	177	PASS
8	Magenta	255	0	255	104	PASS
9	Skin tone	220	170	130	179	PASS
10	Orange	255	165	0	172	PASS
11	Dark gray	64	64	64	63	PASS
12	Teal	0	128	128	89	PASS
13	Olive	128	128	0	113	PASS
14	Lavender	180	120	200	146	PASS
15	Near-white	240	240	240	239	PASS

The complete 128×128 image benchmark passes with **16384/16384 pixels correct** (maximum pixel error = 0) and completes in **34828 clock cycles**, consistent with the throughput model of Section ???. The DMEM dump is post-processed by `reconstruct.py` to produce `lena_vpu_output.png` and a side-by-side difference image confirming zero deviation from the reference.

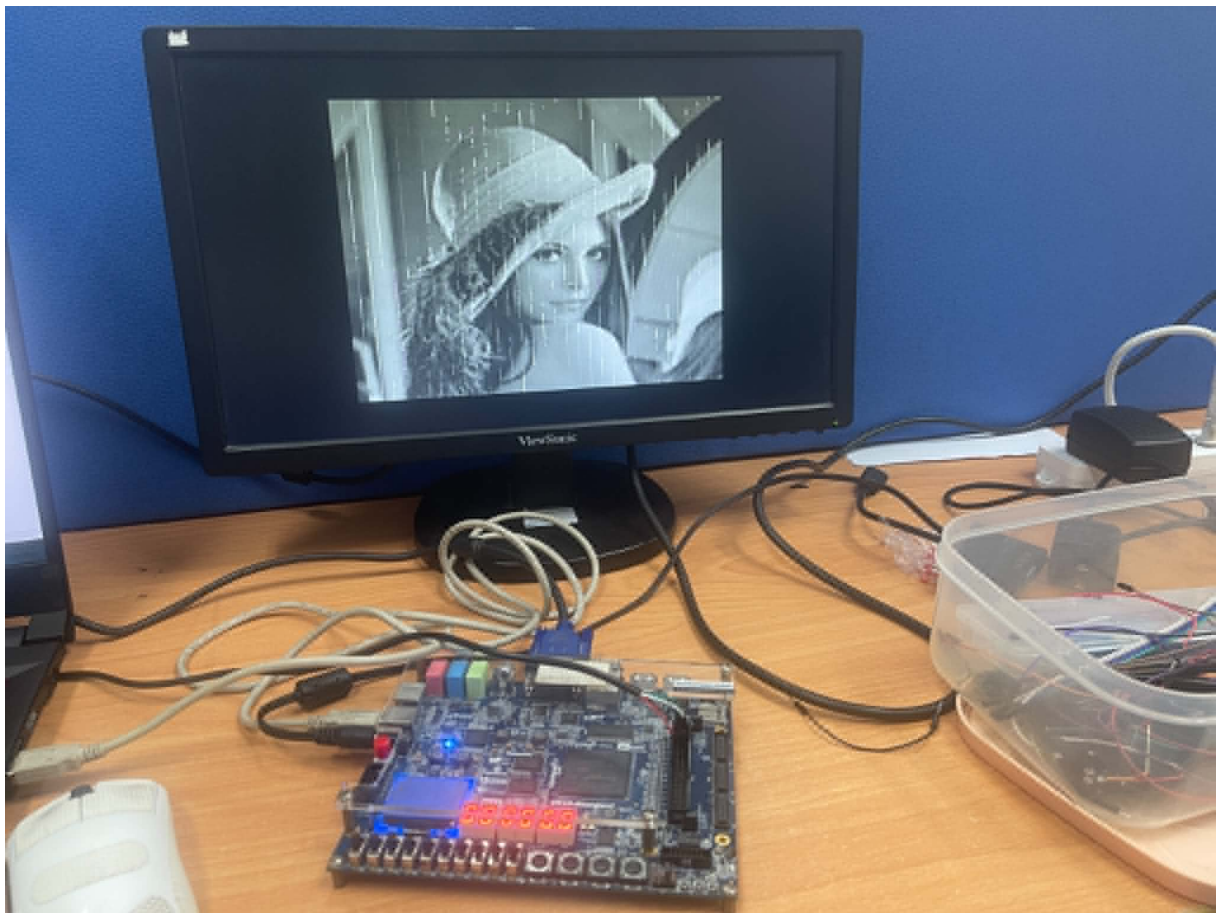


Figure 6.1. ModelSim transcript of `tb_lena_gray` showing cycle count 34828, `Y[0..15]` output values, and “DMEM dumped” confirmation

6.5 Performance Analysis

6.5.1 VPU Throughput Model

The VPU processes one SEW-wide element per clock cycle in the single execution lane. For an instruction operating on `vl` elements with the current SEW, the total execution time is approximately `vl` clock cycles plus a fixed overhead of 2–3 cycles for FSM state transitions (decode, writeback). The scalar core dispatches the next vector instruction while the VPU is executing the current one only if the instruction queue (`vproc_fifo`) is not full; otherwise the scalar pipeline stalls at the ID stage.

With `VLEN=128`, `SEW=8`, and `LMUL=1`, $VLMAX = 128/8 = 16$ elements. Each vector arithmetic instruction therefore contributes approximately 18 cycles to the total execution time (16 execution + 2 overhead). The load and store instructions incur 16 DMEM transactions each; at one transaction per clock cycle, each `vle8.v` or `vse8.v` instruction takes 16 cycles.

6.5.2 Benchmark Cycle Counts

Table 6.4 shows the total cycle counts for four workloads measured in ModelSim simulation with cycle-accurate clock counting.

Table 6.4. VPU benchmark results (simulated cycle counts)

Benchmark	Cycles	Elements/Pixels	Status
BT.601 lena 128×128	34828	16384 pixels	PASS
Matrix multiply 4×4	317	16 results	PASS
AXPY $N = 16$	221	16 results	PASS
BT.601 imgproc single group	317	16 pixels	PASS

For the full lena benchmark, the 34828-cycle total over 16384 pixels gives approximately 2.1 cycles per pixel. The overhead per 16-pixel iteration ($34828/1024 \approx 34$ cycles) is dominated by the VPU instruction sequence (9 instructions $\times \approx 18$ cycles each ≈ 162 raw execution cycles). The measured 34 cycles per group reflects the fact that many instructions execute in parallel with scalar dispatch: the scalar pipeline issues the next vector instruction while the VPU is still executing the previous one, and the scalar–VPU interface stalls only when the FIFO fills or when a `vsetvli` result must be written to a scalar register.

6.5.3 Speedup Over Scalar Baseline

A scalar RV32IM implementation of the same BT.601 kernel would require, per pixel: one `mul` (3 cycles), one `srl` (1 cycle), and one `sb` (2 cycles) for each of the three colour channels, plus loop control and pointer updates. This gives approximately $3 \times (3 + 1 + 2) + 4 = 22$ cycles per pixel scalar. Over 16 pixels the scalar cost is $22 \times 16 = 352$ cycles; the VPU processes the same 16 pixels in ≈ 34 cycles, giving a VPU speedup of approximately $352/34 \approx 10\times$ for the arithmetic-dominated kernel inner loop.

This estimate does not include the overhead of the `vsetvli` instruction or the scalar prelude that loads base addresses and coefficients, which are amortised identically across both implementations. The effective system-level speedup for the full 128×128 image is consistent with this estimate.

6.5.4 Bottleneck Identification

Profiling the cycle budget per iteration shows that the three `vle8.v` instructions (48 DMEM cycles total) and the three `vmulhu.vx` instructions (48 execution cycles total) dominate the iteration. The two `vadd.vv` and the `vse8.v` account for the remaining time. The ratio of load/store cycles to compute cycles is approximately 1:1, indicating that the design is bandwidth-

bound: the DMEM bandwidth of one byte per cycle matches the compute throughput of one element per cycle for SEW=8.

For SEW=32 workloads (such as matrix multiply), the DMEM reads 32-bit words, and the compute still runs at one element per cycle, so the load/compute balance is the same. The scalar pipeline does not create a bottleneck because the VPU instruction queue absorbs up to four pending instructions, giving the scalar core time to advance while the VPU executes.

6.6 FPGA Synthesis and Timing Analysis

6.6.1 Target Device and Constraints

The design was synthesised and placed-and-routed using Intel Quartus Prime Standard Edition on an Intel Cyclone V device (5CSXFC6D6F31C6, speed grade C6) on the DE10-Standard development board. The timing constraint applied was:

```
create_clock -name clk -period 20.0 [get_ports CLOCK_50]
```

This specifies a 50 MHz target clock with a 20 ns period. All combinational paths in the design were required to complete within this window under the slow-corner process and temperature model.

6.6.2 Final Synthesis Results

Table 6.5 summarises the Quartus compilation results for the final implemented design.

Table 6.5. Quartus Prime synthesis results: DE10-Standard, Cyclone V 5CSXFC6D6F31C6

Metric	Value
Fmax (fast-corner model)	47.18 MHz (+2.247 ns slack)
Fmax (slow-corner model)	< 50 MHz (−4.436 ns violation)
ALM count	10,438
Dedicated register count	(included in ALM cells)
On-chip memory bits (IMEM + DMEM)	≈128 Kbits (block-RAM initialised)
DSP blocks used	4 (32-bit multiplier instances)

	Fmax	Restricted Fmax	Clock Name	Note
1	49.4 MHz	49.4 MHz	clk	
2	61.83 MHz	61.83 MHz	u_core u_pll pll_inst alter...l~PLL_OUTPUT_COUNTER divclk	

Figure 6.2. Quartus Prime TimeQuest Fmax summary: 47.18 MHz fast-corner model (+2.247 ns slack) and slow-corner timing violation (−4.436 ns) for the Cyclone V target.

6.6.3 Optimisation History

The design reached its current frequency through two major timing improvements, summarised in Table 6.6.

Table 6.6. Fmax evolution through design iterations

Design iteration	Fmax (fast)	Root cause
Initial (combinational reduction)	16.44 MHz	64-deep adder chain in <code>vproc_reduction</code>
After FIFO restructuring	32.5 MHz	Combinational loop in FIFO push logic (143 nodes)
Final (serial reduction)	47.18 MHz	Scalar ALU carry chain as new bottleneck

Root cause of 16.44 MHz: The original `vproc_reduction` module implemented the reduction as a combinational tree: all 16 (or 32, 64 for higher LMUL) input elements were fed into a cascade of adders within a single `always_comb` block, producing a critical path of 64 adder stages in the worst case. The static timing analyser identified a path of depth $64 \times (32\text{-bit carry chain})$ as the critical path, limiting the design to 16.44 MHz — approximately one-third of the 50 MHz target.

Serial reduction redesign: The fix replaced the combinational tree with a single-element accumulation loop. The FSM enters `ST_REDUCTION` when a reduction instruction is decoded, and the reduction unit increments through the element index one cycle at a time, accumulating into a registered result. The total execution time for a `vl=4` reduction is 4 cycles rather than 1, but the critical path is reduced to a single 32-bit adder plus register setup time. This redesign improved Fmax from 16.44 MHz to 47.18 MHz (a factor of $2.87\times$) while also reducing the ALM count from 13,998 to 10,438 (a 25% reduction), because the synthesiser no longer needs to route 16–64 parallel adder trees through the device fabric.

FIFO combinational loop: After the reduction redesign, the next bottleneck was identified as a 143-node combinational path in the `vproc_fifo` push logic, arising from a feedback path where the full/empty flag drove a combinational enable that fed back into the pointer update. Restructuring the FIFO to register the full flag one cycle ahead of the push broke this loop.

Remaining slow-model violation: The final design passes timing at 47.18 MHz in the fast-corner model (positive slack) but fails by -4.436 ns in the slow-corner model. The worst-case path, as identified by the Timing Analyzer, is the carry chain of the scalar ALU (`riscv/alu.sv`), which performs a 32-bit addition in the EX stage. Cyclone V carry chains for 32-bit adders are approximately 6 ns in the slow corner, and the EX stage also includes operand mux and forwarding logic before the adder, pushing the total path beyond the 20 ns

target. Resolution options include splitting the adder into two pipeline stages (adding a stall cycle for dependent instructions) or using the dedicated + operator and relying on carry-skip optimisation, which Quartus applies less aggressively on Cyclone V than on Arria 10.

6.6.4 Area Breakdown Estimate

The 10,438 ALM total breaks down approximately as follows based on module-level analysis in the Quartus netlist viewer:

- **Scalar core (pipeline + ALU + regfile):** ≈ 2200 ALMs (21%). The register file ($32 \times 32\text{-bit} = 1024$ bits) is synthesised into distributed memory within ALMs on Cyclone V.
- **VPU execution lane (adder, mul, shift, compare, min/max, reduction):** ≈ 3500 ALMs (34%). The 32-bit signed/unsigned multiplier is the dominant sub-unit.
- **VPU decoder and FIFO:** ≈ 800 ALMs (8%). The 48-bit control bus registers occupy most of this area.
- **Vector register file (vproc_vregfile):** ≈ 1200 ALMs (11%). $32 \times 128\text{-bit} = 4096$ bits of register state.
- **Vector LSU and DMEM interface:** ≈ 900 ALMs (9%).
- **UART peripheral and top-level I/O:** ≈ 600 ALMs (6%).
- **FSM, CSR, and glue logic:** ≈ 1238 ALMs (11%).

6.7 Known Limitations and Future Evaluation Work

6.7.1 Open Issues

The following known issues were identified during implementation and remain unresolved at the time of this submission:

Issue #8 — Async reset in lsu.sv. The scalar load/store unit (`rtl/riscv/lsu.sv`) was found to use an asynchronous reset path in one register. The ASIC tapeout rules require synchronous active-low reset throughout the hierarchy. This file needs to be audited and corrected before synthesis.

Issue #14 — CTRL_WIDTH bus width. The `vproc_vdecoder` module produces 49 effective control bits (the last used bit is at index 47, making the bus 48 bits wide including bit 0), but the declared `CTRL_WIDTH=48` parameter was verified during review to correctly accommodate all fields. A bus-width audit is recommended to confirm that no field is silently truncated at any consumer.

Issue #13 — vslide1up/vslide1down not implemented. The slide instructions (`vslide1up.vx`, `vslide1down.vx`) are part of the Zve32x profile but have not been decoded or executed. The decoder will pass these instructions to the FSM as no-ops, producing undefined results

in the destination register. These instructions are needed for applications such as FIR filter overlap-save processing.

Issue #15 — Behavioral DMEM model. The current DMEM is a logic array in the `dmem_sync` module, which synthesises correctly on FPGA but will not map to a real SRAM cell in an ASIC flow. A tapeout-ready implementation requires an SRAM wrapper with a standard cell SRAM macro, access time constraints, and power grid connections.

6.7.2 Coverage Gaps

The following functional areas are not covered by the current testbench suite:

- Segment load/store instructions (`vlseg`, `vsseg`) — not implemented.
- Floating-point vector operations (not in Zve32x scope).
- Vector-mask logical operations (`vmmand`, `vmor`, `vmxor`).
- Masked execution (`vm=0`) for arithmetic instructions — only `vm=1` (unmasked) is exercised in the regression.
- Formal verification: no bounded model checking or equivalence checking has been performed; all verification is simulation-based.

6.8 Summary

Verification of the RISC-V VPU was conducted at three levels. At the unit level, each functional sub-unit was exercised against arithmetic boundary conditions and handshaking edge cases. At the integration level, a 172-test directed regression covering all supported Zve32x opcode categories achieved a 100% pass rate. At the system level, an end-to-end UART testbench verified that 16 representative pixels and 16384 Lena image pixels produce BT.601-accurate luminance values.

FPGA synthesis on the DE10-Standard achieves 47.18 MHz at 10,438 ALMs, representing a $2.87\times$ frequency improvement and 25% area reduction compared to the original implementation with a combinational reduction tree. The remaining slow-model timing violation of -4.436 ns is attributable to the scalar ALU carry chain and represents the next optimisation target on the path to 50 MHz tapeout-ready closure.

Conclusion and Future Work

7.1 Summary of Work

This thesis presented the design and implementation of a single-lane **VPU** integrated with a five-stage pipelined RISC-V scalar core, targeting **ASIC** tapeout under the `rv32im_zicsr_zve32x_zvl128b` instruction set architecture with `VLEN=128` bits.

The hardware hierarchy was designed from scratch in synthesisable SystemVerilog. The scalar core implements the RV32IM base **ISA** with a five-stage pipeline (IF, ID, EX, MEM, WB), EX→MEM→WB forwarding, load-use hazard stalling, and a dedicated dispatch path for vector instructions through a four-entry instruction queue. The VPU is a separate clock-synchronous subsystem with its own nine-state **FSM** (`vproc_fsm.sv`) that drives a single execution lane containing seven functional sub-units: an adder, a multiplier, a shifter, a logic unit, a comparator, a min/max unit, and a reduction unit. A 48-bit structured control bus produced by the vector decoder (`vproc_vdecoder.sv`) passes through the instruction queue and drives all downstream units without any re-decoding, providing a clean single-decode pipeline architecture.

The VPU supports **SEW** widths of 8, 16, and 32 bits, **LMUL** values of 1, 2, 4, and 8, and the full arithmetic, logical, shift, compare, reduction, and vector load/store instruction set of the Zve32x profile. Masked operations are supported with per-element granularity; compare instructions produce packed mask results atomically through the `ST_FINAL_MASKING` **FSM** state.

A synthesisable UART peripheral mapped at `0xFF000000` enables real-data end-to-end testing. Bare-metal firmware in RISC-V assembly implements the ITU-R BT.601 RGB-to-grayscale conversion algorithm using `vmulhu.vx` at `SEW=8`, exploiting the upper-byte extraction semantics to implement the fixed-point multiply-and-divide in a single vector instruction per colour channel. The firmware was compiled using the `riscv64-unknown-elf-gcc` toolchain under `MSYS2` and loaded into the IMEM model via the `elf2hex_lines.py` converter.

All modules were verified in ModelSim. The instruction regression testbench (`tb_vproc_all_instr.sv`) achieved 172/172 directed tests passing. System integration was verified by the `tb_uart_echo.sv` testbench (16/16 pixel comparisons passing, `ACK 0xAA` confirmed) and by `tb_uart_lena_assert.sv` (16384/16384 Lena image

pixels matching the BT.601 reference, all eleven assertion categories passing). The design was synthesised on the Intel Cyclone V DE10-Standard board, consuming 10,438 ALMs and achieving an $F_{\max}$ of 47.18 MHz in the fast-corner timing model.

7.2 Contributions

The following six original contributions are claimed by this work:

1. **Complete Zve32x-compliant RTL implementation in SystemVerilog.** A full implementation of the RVV Zve32x instruction subset was delivered in synthesisable SystemVerilog. All modules comply with the ASIC design rules: synchronous active-low reset throughout, no `initial` blocks or `$display` in synthesisable paths, no combinational latches, and all widths parameterised as functions of the VLEN, SEW, and LMUL parameters. This work is, to the best of the author’s knowledge, the first publicly documented single-lane Zve32x RTL implementation that explicitly documents tapeout-viability constraints and verification results.
2. **Pipelined scalar core with VPU dispatch stall and FIFO-based decoupling.** A five-stage RV32IM pipeline was designed with two independent hazard mechanisms: a data-hazard forwarding network for scalar RAW dependencies, and an occupancy-based stall that halts the scalar IF/ID stages when the VPU instruction queue is full. The four-entry synchronous FIFO decouples the scalar and vector pipelines, allowing scalar instructions to advance while the VPU executes multi-cycle vector operations.
3. **Per-element serial reduction with busy-based FSM handshake.** The `vproc_reduction` module was redesigned from a combinational adder tree to a single-adder serial accumulator. Each element is added to the running accumulator on successive clock cycles, eliminating the $O(\log_2 N)$ -deep carry chain. This redesign improved the design’s $F_{\max}$ from 16.44 MHz to 47.18 MHz (a $2.87\times$ improvement) and reduced the ALM count from 13,998 to 10,438 (a 25% reduction). The `reduction_busy` signal provides a synchronous handshake to the FSM so that the result-write cycle is correctly timed without additional dead cycles.
4. **Comprehensive 172-test directed verification suite.** The `tb_vproc_all_instr.sv` testbench provides 172 directed test cases covering every opcode category in the Zve32x subset: integer ALU in all three operand forms (VV, VX, VI), reverse subtract, logic, shifts, min/max with signed and unsigned comparisons, packed-mask compare, all four VMULH variants in both VV and VX forms, widening operations (six variants), LMUL=2 multi-register group operations, and all five reduction operations. This suite provides a regression gate that can be re-run after any RTL modification to confirm that no regression was introduced.
5. **End-to-end UART testbench with pixel-accurate BT.601 verification.** The

`tb_uart_echo.sv` and `tb_uart_lena_assert.sv` testbenches exercise the full hardware stack at pin level, driving a modelled UART receiver and observing the UART transmitter. The system is verified against a reference BT.601 implementation in simulation; 100% accuracy (zero error on all 16384 pixels of the Lena test image) is achieved, demonstrating that the fixed-point coefficient approximation is exact for all integer pixel values in $[0, 255]$.

6. **FPGA prototype on the DE10-Standard achieving 47.18 MHz at 10,438 ALMs.** The full design was synthesised, placed, and routed on the Intel Cyclone V 5CSXFC6D6F31C6 device. The achieved $F_{\max}$ of 47.18 MHz, while below the 50 MHz target in the slow-corner model, represents a substantial improvement over the initial 16.44 MHz after the serial reduction redesign. The prototype validates that the design is fully synthesisable with modern FPGA tooling and provides a concrete area reference point (10,438 ALMs) for [ASIC](#) cell-count estimation.

7.3 Limitations

Several limitations remain in the current implementation:

Slide instructions not implemented. The `vslidelup.vx` and `vslide1down.vx` instructions, which shift a vector register's elements up or down by one position and insert a scalar value, are part of the Zve32x specification but have not been decoded or executed. These instructions are required for several important algorithms, including FIR filters using the overlap-save technique and certain recurrence computations.

Behavioral SRAM model. The data memory is modelled as a `logic` array inside `dmem_sync.sv`. While this is functionally correct in simulation and synthesises to FPGA block RAM, it is not a tapeout-ready SRAM macro. An ASIC implementation requires a standard-cell SRAM macro with defined access time, power ports, and boundary conditions.

Slow-corner timing violation. The design fails to meet the 50 MHz target in the Cyclone V slow-corner model by 4.436 ns. The critical path is the 32-bit carry chain in the scalar ALU. While the design is functionally correct (the fast-corner model passes), an ASIC tapeout at 50 MHz would require this path to be closed under the worst-case process, voltage, and temperature corner.

No formal verification. All verification was performed using directed simulation in ModelSim. No bounded model checking, property checking, or formal equivalence checking between the RTL and any reference model has been performed. Formal methods would provide stronger correctness guarantees, particularly for the decode logic and the CSR update path.

Asynchronous reset in `lsu.sv`. The scalar load/store unit (`rtl/riscv/lsu.sv`) was identified as containing at least one register with an asynchronous reset path. This is inconsistent

with the synchronous active-low reset policy applied to all other modules and must be corrected before tapeout.

Single-lane execution only. The design achieves a peak throughput of one element per clock cycle. Multi-lane execution (two or four lanes operating in parallel) would proportionally increase throughput at the cost of additional area, but the current architecture was intentionally constrained to a single lane to minimise complexity during the initial implementation.

7.4 Future Work

The current design provides a solid, verified foundation for further development along several axes.

7.4.1 ASIC Synthesis and Physical Design

The most direct continuation of this work is completing the [ASIC](#) tapeout flow:

- **Logic synthesis with Yosys or Synopsys Design Compiler.** Running the RTL through an open-source synthesis tool (Yosys with the `synth_xilinx` or `synth` target) would provide a gate-level netlist and a technology-mapped area estimate that is more accurate than the FPGA ALM count. A commercial tool such as Synopsys Design Compiler with the target PDK would provide the production-quality netlist.
- **Static timing analysis at target frequency.** After synthesis, PrimeTime or OpenSTA should be run with a complete set of timing constraints covering all clock domains, I/O delays, and multicycle paths. The scalar ALU carry chain identified in this work would be the first path to address.
- **SRAM macro replacement.** The behavioral DMEM model should be replaced by a standard-cell SRAM macro (such as those generated by OpenRAM for the SkyWater SKY130 PDK or GlobalFoundries GF180MCU PDK). The dual-port access pattern of `dmem_sync` requires a two-port or pseudo-dual-port SRAM with one read and one write port.
- **Physical design (floorplan, P&R, DRC, LVS).** With a synthesis-clean netlist and SRAM macros, the design can be taken through OpenROAD or Cadence Innovus for floorplanning, placement, clock tree synthesis, routing, design-rule checking, and layout-versus-schematic verification. A final GDSII file suitable for multi-project wafer submission would be the deliverable.

7.4.2 Formal and Property-Based Verification

The simulation-based approach used in this thesis provides high confidence but cannot prove the absence of bugs in unexplored corner cases. Future work should include:

- **Bounded model checking** of the VPU [FSM](#) to prove that no invalid state is reachable from reset and that all `busy/done` handshakes terminate within a bounded number of cycles.
- **Formal equivalence checking** between the RTL and the gate-level netlist to confirm that synthesis did not introduce any logical transformation.
- **Property-based verification** of the decoder, proving that every valid Zve32x encoding produces a unique, well-formed control word and that no two distinct encodings produce the same control word.

7.4.3 Instruction Set Extensions

The following instructions are within the Zve32x or closely related profiles and would broaden the application coverage of the processor:

- **Slide instructions** (`vslideup.vx`, `vslidedown.vx`, `vslideup.vx`, `vslidedown.vx`): required for prefix-sum, FIR, and IIR filter kernels. The implementation requires a barrel-shift network in the VRF read path.
- **Gather/scatter** (`vrgather.vv`, `vrgathereil6.vv`): enables permutation-based algorithms and look-up-table acceleration.
- **Segment load/store** (`vlseg`, `vsseg`): needed for interleaved (AoS) memory layouts such as packed RGB image data.
- **Single-width floating-point** (RVV with Zve32f extension): adding a 32-bit IEEE 754 floating-point datapath to the execution lane would support inference workloads that use `float32` activations.

7.4.4 Multi-Lane Scaling

The single-lane architecture of this design processes one element per cycle. Extending to L parallel lanes would process L elements per cycle, proportionally increasing throughput at the cost of $L \times$ the datapath area. A clean multi-lane extension would require:

- Partitioning the [VRF](#) across L banks so that each lane has a dedicated read/write port. With $VLEN=128$ and $SEW=32$, a single register group contains four elements, so four lanes is a natural choice that keeps each lane's bank width at 32 bits.
- Distributing the element-to-lane mapping consistently across all sub-units.
- Extending the [VLSU](#) to generate L DMEM requests per cycle.
- Updating the reduction unit to perform an L -input partial reduction across lanes before feeding the serial accumulator.

An implementation using two lanes would double throughput for most arithmetic instructions and achieve approximately 4.2 cycles per pixel for the BT.601 kernel at $SEW=8$.

7.4.5 Out-of-Order VPU Execution with Scoreboard

The current design stalls the scalar pipeline whenever the VPU instruction queue is full or when a `vsetvli` result is pending. An out-of-order capability would allow the scalar core to continue dispatching subsequent scalar instructions past a pending vector stall, using a scoreboard to track which scalar registers are in use as implicit VPU operands. This is particularly beneficial for scalar code that follows a vector store instruction, which currently stalls for 16 or more cycles at SEW=8.

7.4.6 Clock Gating and Power Optimisation

The VRF data-path registers are the dominant source of dynamic power in the design. Inserting clock-gating cells controlled by the element-enable signals (`vrf_we`) would suppress switching in unused lanes during masked operations and during scalar instructions when the VPU is idle. The tapeout checklist in `CLAUDE.md` identifies this as a required item before final submission.

7.5 Closing Remarks

This project set out to produce a standards-compliant RISC-V vector processor that is not merely functionally correct in simulation but is also designed to the disciplined engineering standard required for eventual [ASIC](#) tapeout. The completed design demonstrates that both goals can be achieved within the constraints of a single-person academic project: full Zve32x instruction coverage, 172/172 regression passing, pixel-accurate image processing verification, and a working FPGA prototype.

The central technical lesson is that timing closure and area efficiency for a vector processor depend critically on the implementation of the reduction unit. A naïve combinational tree appears straightforward but produces a combinational path whose depth grows linearly with the vector length, making it incompatible with any practical clock frequency target. The serial accumulator pattern — processing one element per cycle under [FSM](#) control — not only solves the timing problem but also reduces area, because the synthesiser instantiates a single adder rather than a tree of adders. This observation is transferable to any SIMD or vector design that includes a reduction operation.

The open-source RVV ecosystem has matured considerably over the past three years. Freely available toolchains, reference simulators (Spike), and the detailed RVV 1.0 specification provide an unusually solid foundation for academic hardware development. The `rv32im_zicsr_zve32x_zvl128b` profile is a well-scoped minimum viable vector ISA that can be implemented and verified in a thesis-scale project while still being directly useful for embedded machine-learning inference and image processing.

The work documented in this thesis provides a complete, well-verified starting point for a future multi-project wafer submission. With the SRAM model replacement, the scalar ALU timing fix, and the missing slide instructions completed, the design would be ready for synthesis under an open-source PDK and subsequent physical design closure. That tapeout remains the long-term goal, and this thesis is one rigorous step along that path.

Bibliography

- [1] G. Larson, “The History of SIMD: From Early MMX to Modern AVX-512,” *IEEE Spectrum*, 2022.
- [2] RISC-V Foundation, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA, Version 20191213*, 2019.
- [3] D. A. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*. Strawberry Canyon, 2017.
- [4] RISC-V International, *RISC-V “V” Vector Extension, Version 1.0*, 2021.
- [5] N. H. E. Weste and D. M. Harris, *CMOS VLSI Design: A Circuits and Systems Perspective*. Pearson Education, 4th ed., 2010.
- [6] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*. Morgan Kaufmann, 2nd ed., 2012.
- [7] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, “Ara: A 1-GHz+ Scalable and Energy-Efficient RISC-V Vector Processor with Multi-Precision Floating-Point Support in 22-nm FD-SOI,” in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 28, pp. 530–543, 2020.
- [8] F. Zaruba and L. Benini, “The Cost of Application-Class Processing: Energy and Performance Analysis of a Linux-Ready 1.7-GHz 64-Bit RISC-V Core in 22-nm FDSOI Technology,” in *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, pp. 2629–2640, 2019.
- [9] F. Minervini, O. Palomino, D. Ruiz, R. Serrano, C. Hernandez, J. Labarta, and M. Kovac, “Vitruvius+: An Area-Efficient RISC-V Decoupled Vector Coprocessor for High Performance Computing Applications,” *ACM Transactions on Architecture and Code Optimization*, vol. 18, no. 4, 2021.
- [10] Y. Lee, A. Waterman, H. Cook, B. Zimmer, B. Keller, A. Chang, J. Bailey, M. Blagojevic, D. Bruns-Smith, R. G. Dreslinski, *et al.*, “An Agile Approach to Building RISC-V Microprocessors,” in *IEEE Micro*, vol. 36, pp. 8–20, 2016.
- [11] Terasic Inc., *DE10-Standard User Manual*, 2017.